

JAVA 17 OCP · YDS STUDY SERIES

Unit 06 · Class Design · Complete Bilingual Notes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu düzenlenebilir ana kaynak, verilen PDF bölümünün sayfa sırasını eksiksiz izler. PDF satır kaymalarından bölünen cümleler mümkün olduğunda özgün paragraf akışında birleştirilmiş; her English paragrafın hemen altına doğal ve teknik Türkçe karşılığı yerleştirilmiştir. Java code, identifier, literal ve exact output aynen korunur; prose soru seçenekleri teknik Türkçeye çevrilir.

[Vocabulary](#) · [Grammar notes](#)

Kapsam manifesti

Alan	Değer
Bölüm	Chapter 6 — Class Design
Kaynak PDF sayfaları	0275–0344
Beklenen marker sayısı	70
Korunan içerik	Başlıklar, paragraflar/satırlar, kod, tablolar, şekiller, callout'lar, Summary, Exam Essentials ve Review Questions
Çıkarılan içerik	Yalnız tekrarlanan running header/footer ve sayfa numarası
Kaynak dışı içerik	Belgenin sonundaki açıkça etiketlenmiş teknik pekiştirme appendix'i

İçindekiler

- [Kapsam manifesti](#)
- [Kaynak metin ve çeviri](#)
- [Understanding Inheritance](#)
- [Creating Classes](#)
- [Declaring Constructors](#)
- [Initializing Objects](#)
- [Inheriting Members](#)
- [Creating Abstract Classes](#)
- [Creating Immutable Objects](#)
- [Summary](#)
- [Exam Essentials](#)
- [Review Questions](#)
- [Page coverage ledger](#)
- [Teknik pekiştirme appendix'i](#)

Kaynak metin ve çeviri

Source page 0275

English: Chapter **Türkçe:** Chapter

English: 6 **Türkçe:** 6

Class Design

Türkçe başlık: Class Tasarımı

OCP EXAM OBJECTIVES COVERED IN

Türkçe başlık: OCP SINAVININ HEDEFLERİ

English: THIS CHAPTER: **Türkçe:** BU BÖLÜM:

English: [x] [x] Utilizing Java Object-Oriented Approach **Türkçe:** [x] [x] Java Object-Oriented Approach (Java nesne yönelimli yaklaşım) kullanımı

English: ■ ■ ■ ■ ■ Create classes and records, and define and use instance and static fields and methods, constructors, and instance and static initializers Understand variable scopes, use local variable type inference, apply encapsulation, and make objects immutable Implement polymorphism and differentiate object type versus reference type. Perform type casting, identify object types using instanceof operator and pattern matching **Türkçe:** ■ ■ ■ ■ ■ Class'lar ve record'lar oluşturun; instance ve static field'ları, method'ları, constructor'ları ve instance/static initializer'ları tanımlayıp kullanın. Variable scope'ları anlayın, local variable type inference kullanın, encapsulation uygulayın ve object'leri immutable hale getirin. Polymorphism uygulayın; object type ile reference type arasındaki farkı belirleyin. Casting yapın; instanceof operator'ü ve pattern matching kullanarak object type'lerini saptayın.

Source page 0276

English: In Chapter 1, “Building Blocks,” we introduced the basic definition of a class in Java. In Chapter 5, “Methods,” we delved into methods and modifiers and showed how you can use them to build more structured classes. In this chapter, we take things a step further and show how class structure and inheritance is one of the most powerful features in the Java language. **Türkçe:** Chapter 1, “Building Blocks”ta Java'da bir class'ın temel tanımını ele aldık. Chapter 5, “Methods”ta method'ları ve modifier'ları ayrıntılı biçimde inceleyip bunlarla daha düzenli class'ların nasıl oluşturulacağını gösterdik. Bu bölümde bir adım daha ileri giderek class yapısının ve inheritance'ın Java dilindeki en güçlü özelliklerden biri olduğunu göreceğiz.

English: At its core, proper Java class design is about code reusability, increased functionality, and standardization. For example, by creating a new class that extends an existing class, you may gain access to a slew of inherited primitives, objects, and methods, which increases code reuse. **Türkçe:** İyi bir Java class tasarımının temelinde code reusability (kodun yeniden kullanılabilirliği), daha fazla işlevsellik ve standardization bulunur. Örneğin mevcut bir class'ı extend eden yeni bir class oluşturduğunuzda, inheritance yoluyla birçok primitive'e, object'e ve method'a erişebilirsiniz; böylece kodun yeniden kullanımı artar.

English: This chapter is the culmination of some of the most important topics in Java including inheritance, class design, constructors, order of initialization, overriding methods, abstract classes, and immutable objects. Read this chapter carefully and make sure you understand all of the topics well. This chapter forms the basis of Chapter 7, “Beyond Classes,” in which we expand our discussion of types to include other top-level and nested types. **Türkçe:** Bu bölüm; inheritance, class design, constructor'lar, initialization order, method overriding, abstract class'lar ve immutable object'ler gibi Java'nın en önemli konularını bir araya getirir. Bölümü dikkatle okuyun ve her konuyu iyice anladığınızdan emin olun. Buradaki bilgiler, type tartışmasını başka top-level ve nested type'ları da kapsayacak biçimde genişleten Chapter 7, “Beyond Classes” için temel oluşturur.

Understanding Inheritance

Türkçe başlık: Inheritance'ı Anlamak

English: When creating a new class in Java, you can define the class as inheriting from an existing class. Inheritance is the process by which a subclass automatically includes certain members of the class, including primitives, objects, or methods, defined in the parent class. **Türkçe:** Java'da yeni bir class oluştururken bu class'ın mevcut bir class'tan inheritance alacağını belirtebilirsiniz. Inheritance; bir subclass'ın parent class'ta tanımlanan primitive, object veya method gibi belirli member'ları otomatik olarak edinmesi sürecidir.

English: For illustrative purposes, we refer to any class that inherits from another class as a subclass or child class, as it is considered a descendant of that class. Alternatively, we refer to the class that the child inherits from as the superclass or parent class, as it is considered an ancestor of the class. **Türkçe:** Açıklamalarda başka bir class'tan inheritance alan class'a, o class'ın descendant'ı (soyundan geleni) olduğu için subclass veya child class deriz. Child class'ın inheritance aldığı class ise onun ancestor'ı (atası) sayıldığından superclass ya da parent class olarak adlandırılır.

English: When working with other types, like interfaces, we tend to use the general terms subtype and supertype. You see this more in the next chapter. **Türkçe:** Interface gibi başka type'larla çalışırken daha genel olan subtype ve supertype terimlerini kullanırız. Bunları sonraki bölümde daha ayrıntılı göreceksiniz.

Declaring a Subclass

Türkçe başlık: Bir Subclass Bildirmek

English: Let's begin with the declaration of a class and its subclass. Figure 6.1 shows an example of a superclass, Mammal, and subclass Rhinoceros. **Türkçe:** Bir class ile subclass'ının declaration'ından başlayalım. Şekil 6.1'de superclass olan `Mammal` ile subclass olan `Rhinoceros` gösterilmektedir.

Source page 0277

English: FIGURE 6.1 Subclass and superclass declarations **Türkçe:** ŞEKİL 6.1 Subclass ve superclass declaration'ları

```
public class Mammal { }
```

English: Superclass **Türkçe:** Superclass

English: public or package access final keyword (optional) class keyword **Türkçe:** `public` veya package access · `final` keyword'ü (isteğe bağlı) · `class` keyword'ü

English: Class **Türkçe:** Class

English: name Extends parent class **Türkçe:** Class adı · parent class'ı `extends` ile belirtme

```
public final class Rhinoceros extends Mammal { }
```

English: Subclass **Türkçe:** Subclass

English: We indicate a class is a subclass by declaring it with the `extends` keyword. We don't need to declare anything in the superclass other than making sure it is not marked `final`. More on that shortly.

Türkçe: Bir class'ın subclass olduğunu declaration'daki `extends` keyword'üyle belirtiriz. Superclass tarafında, class'ın `final` olmadığından emin olmak dışında özel bir declaration yapmamız gerekmez. `final` konusuna birazdan döneceğiz.

English: One key aspect of inheritance is that it is transitive. Given three classes [X, Y, Z], if X extends Y, and Y extends Z, then X is considered a subclass or descendant of Z. Likewise, Z is a superclass or ancestor of X. We sometimes use the term direct subclass or descendant to indicate the class directly extends the parent class. For example, X is a direct descendant only of class Y, not Z. **Türkçe:** Inheritance'ın temel özelliklerinden biri transitive (geçişli) olmasıdır. [X, Y, Z] class'larında X, Y'yi; Y de Z'yi extend ediyorsa X, Z'nin subclass'ı/descendant'ı sayılır. Benzer biçimde Z, X'in superclass'ı/ancestor'ıdır. Bir class'ın parent class'ı doğrudan extend ettiğini vurgulamak için direct subclass veya direct descendant deriz. Örneğin X yalnızca Y'nin direct descendant'ıdır; Z'nin değildir.

English: In the last chapter, you learned that there are four access levels: public, protected, package, and private. When one class inherits from a parent class, all public and protected members are automatically available as part of the child class. If the two classes are in the same package, then package members are available to the child class. Last but not least, private members are restricted to the class they are defined in and are never available via inheritance. This doesn't mean the parent class can't have private members that can hold data or modify an object; it just means the subclass doesn't have direct access to them. **Türkçe:** Önceki bölümde dört access level olduğunu öğrendiniz: `public`, `protected`, package ve `private`. Bir class parent class'tan inheritance aldığı anda bütün `public` ve `protected` member'lar child class tarafından kullanılabilir. İki class aynı package'taysa package-access member'lar da child class'a açıktır. `private` member'lar ise yalnızca tanımlandıkları class içinde erişilebilir; inheritance yoluyla doğrudan erişilemez. Bu, parent class'ın veri saklayan veya object'i değiştiren `private` member'lara sahip olamayacağı anlamına gelmez; yalnızca subclass'ın bunlara doğrudan erişimi yoktur.

English: Let's take a look at a simple example: **Türkçe:** Basit bir örneğe bakalım:

```
public class BigCat {
    protected double size;
}
public class Jaguar extends BigCat {
    public Jaguar() {
        size = 10.2;
    }
    public void printDetails() {
        System.out.print(size);
    }
}
```

Source page 0278

```
public class Spider {
    public void printDetails() {
        System.out.println(size); // DOES NOT COMPILE
    }
}
```

English: Jaguar is a subclass or child of BigCat, making BigCat a superclass or parent of Jaguar. In the Jaguar class, size is accessible because it is marked protected. Via inheritance, the Jaguar subclass can read or write size as if it were its own member. Contrast this with the Spider class, which has no access to size since it is not inherited. **Türkçe:** Jaguar, BigCat'in subclass'ı/child'ıdır; dolayısıyla BigCat, Jaguar'ın superclass'ı/parent'ıdır. size, protected olduğu için Jaguar class'ından erişilebilir. Inheritance sayesinde Jaguar, size değerini kendi member'ıymış gibi okuyup yazabilir. Buna karşılık Spider, size member'ını inherit etmediğinden ona erişemez.

Class Modifiers

Türkçe başlık: Class Modifier'ları

English: Like methods and variables, a class declaration can have various modifiers. Table 6.1 lists the modifiers you should know for the exam. **Türkçe:** Method ve variable'larda olduğu gibi, bir class declaration'ında da çeşitli modifier'lar bulunabilir. Tablo 6.1 sınav için bilmeniz gereken modifier'ları listeler.

English: TABLE 6.1 Class modifiers **Türkçe:** TABLO 6.1 Class modifier'ları

English: Modifier · Description · Chapter covered **Türkçe:** Modifier · Açıklama · Ele alındığı chapter

English: `final` · The class may not be extended. · Chapter 6 **Türkçe:** `final` · Class extend edilemez. · Chapter 6

English: `abstract` · The class is abstract, may contain abstract methods, and requires a concrete subclass to instantiate. · Chapter 6 **Türkçe:** `abstract` · Class abstract'tır, abstract method içerebilir ve instantiate edilmesi için concrete subclass gerekir. · Chapter 6

English: `sealed` · The class may only be extended by a specific list of classes. · Chapter 7 **Türkçe:** `sealed` · Class yalnızca belirtilen class listesi tarafından extend edilebilir. · Chapter 7

English: `non-sealed` · A subclass of a sealed class permits potentially unnamed subclasses. · Chapter 7 **Türkçe:** `non-sealed` · Sealed class'ın bir subclass'ı, önceden adı belirtilmemiş subclass'lara izin verir. · Chapter 7

English: `static` · Used for static nested classes defined within a class. · Chapter 7 **Türkçe:** `static` · Başka bir class içinde tanımlanan static nested class'larda kullanılır. · Chapter 7

English: We cover abstract classes later in this chapter. In the next chapter, we cover sealed and non-sealed classes, as well as static nested classes. **Türkçe:** Abstract class'ları bu bölümün ilerleyen kısmında ele alacağız. Sealed/non-sealed class'lar ile static nested class'lar ise sonraki bölümün konusudur.

English: For now, let's talk about marking a class final. The final modifier prevents a class from being extended any further. For example, the following does not compile: **Türkçe:** Şimdilik bir class'ı `final` olarak işaretlemeye bakalım. `final` modifier'ı class'ın başka bir class tarafından extend edilmesini engeller. Örneğin aşağıdaki kod derlenmez:

```
public final class Rhinoceros extends Mammal { }  
public class Clara extends Rhinoceros { } // DOES NOT COMPILE
```

English: On the exam, pay attention to any class marked final. If you see another class extending it, you know immediately the code does not compile. **Türkçe:** Sınavda `final` işaretli class'lara dikkat edin. Başka bir class'ın onu extend ettiğini görürseniz kodun derlenmediğini hemen anlayabilirsiniz.

Source page 0279

English: Single vs. Multiple Inheritance **Türkçe:** Single ve Multiple Inheritance

English: Java supports single inheritance, by which a class may inherit from only one direct parent class. Java also supports multiple levels of inheritance, by which one class may extend another class, which in turn extends another class. You can have any number of levels of inheritance, allowing each descendant to gain access to its ancestor's members. **Türkçe:** Java single inheritance'ı destekler; yani bir class yalnızca bir direct parent class'tan inheritance alabilir. Bununla birlikte inheritance birden fazla düzeyden oluşabilir: Bir class başka bir class'ı, o class da üçüncü bir class'ı extend edebilir. Düzey sayısında bir sınır yoktur ve her descendant, ancestor'larının erişilebilir member'larını edinir.

English: To truly understand single inheritance, it may be helpful to contrast it with multiple inheritance, by which a class may have multiple direct parents. By design, Java doesn't support multiple inheritance in the language because multiple inheritance can lead to complex, often difficult-to-maintain data models. Java does allow one exception to the single inheritance rule, which you see in Chapter 7— a class may implement multiple interfaces. **Türkçe:** Single inheritance'ı anlamanın iyi bir yolu, onu bir class'ın birden fazla direct parent'a sahip olabildiği multiple inheritance ile karşılaştırmaktır. Java dili, karmaşık ve bakımı zor veri modellerine yol açabildiği için class'larda multiple inheritance'ı tasarım gereği desteklemez. Chapter 7'de göreceğiniz bir istisna vardır: Bir class birden fazla interface'i implement edebilir.

English: Figure 6.2 illustrates the various types of inheritance models. The items on the left are considered single inheritance because each child has exactly one parent. You may notice that single inheritance doesn't preclude parents from having multiple children. The right side shows items that have multiple inheritance. As you can see, a Dog object has multiple parent designations. **Türkçe:** Şekil 6.2 farklı inheritance modellerini gösterir. Soldaki yapılar single inheritance örneğidir; çünkü her child'ın tam olarak bir parent'ı vardır. Single inheritance, bir parent'ın birden fazla child'a sahip olmasını engellemez. Sağdaki yapılarda ise multiple inheritance vardır: Görüldüğü gibi bir Dog object'i birden fazla parent ile ilişkilendirilmiştir.

English: FIGURE 6.2 Types of inheritance **Türkçe:** ŞEKİL 6.2 Inheritance türleri

Animal Animal Pet Friendly

Türkçe başlık: Animal · Animal · Pet Friendly

Mammal Bird

Türkçe başlık: Mammal Bird

English: Dog **Türkçe:** Dog

Bat Tiger Parrot Eagle Husky Poodle

Türkçe başlık: Bat · Tiger · Parrot · Eagle · Husky · Poodle

Single Inheritance Multiple Inheritance

Türkçe başlık: Single Inheritance · Multiple Inheritance

English: Part of what makes multiple inheritance complicated is determining which parent to inherit values from in case of a conflict. For example, if you have an object or method defined in all of the parents, which one does the child inherit? There is no natural ordering for parents in this example, which is why Java avoids these issues by disallowing multiple inheritance altogether. **Türkçe:** Multiple inheritance'ı zorlaştıran konulardan biri, conflict durumunda değerin hangi parent'tan alınacağını belirlemektir. Örneğin bütün parent'larda aynı adlı bir object ya da method tanımlıysa child hangisini inherit etmelidir? Parent'lar arasında doğal bir öncelik sırası olmadığından Java, class'lar için multiple inheritance'ı bütünüyle yasaklayarak bu sorunları önler.

Inheriting Object

Türkçe başlık: Object'ten Inheritance Alma

English: Throughout our discussion of Java in this book, we have thrown around the word object numerous times— and with good reason. In Java, all classes inherit from a single class: `java.lang.Object`, or `Object` for short. Furthermore, `Object` is the only class that doesn't have a parent class. **Türkçe:** Kitap boyunca object sözcüğünü sıkça kullandık; bunun iyi bir nedeni var. Java'da bütün class'lar tek bir class'tan inheritance alır: `java.lang.Object`, kısaca `Object`. Ayrıca parent class'ı bulunmayan tek class da `Object`'tir.

Source page 0280

English: You might be wondering, “None of the classes I've written so far extend `Object`, so how do all classes inherit from it?” The answer is that the compiler has been automatically inserting code into any class you write that doesn't extend a specific class. For example, the following two are equivalent:

Türkçe: “Şimdiye kadar yazdığım class'ların hiçbiri `Object`'i extend etmiyor; öyleyse bütün class'lar ondan nasıl inheritance alıyor?” diye düşünebilirsiniz. Belirli bir class'ı extend etmeyen her class'a gerekli kodu compiler otomatik ekler. Örneğin aşağıdaki iki declaration eşdeğerdir:

```
public class Zoo { }  
public class Zoo extends java.lang.Object { }
```

English: The key is that when Java sees you define a class that doesn't extend another class, the compiler automatically adds the syntax `extends java.lang.Object` to the class definition. The result is that every class gains access to any accessible methods in the `Object` class. **Türkçe:** Temel nokta şudur: Java başka bir class'ı extend etmeyen bir class declaration'ı gördüğünde compiler, class tanımına otomatik olarak `extends java.lang.Object` ekler. Böylece her class, `Object` class'ındaki erişilebilir method'lara erişir.

English: For example, the `toString()` and `equals()` methods are available in `Object`; therefore, they are accessible in all classes. Without being overridden in a subclass, though, they may not be particularly useful. We cover overriding methods later in this chapter. **Türkçe:** Örneğin `toString()` ve `equals()` method'ları `Object`'te tanımlıdır; dolayısıyla bütün class'larda erişilebilir. Ne var ki subclass'ta override edilmediklerinde çok yararlı olmayabilirler. Method overriding'i bu bölümün ilerleyen kısmında ele alacağız.

English: On the other hand, when you define a new class that extends an existing class, Java does not automatically extend the Object class. Since all classes inherit from Object, extending an existing class means the child already inherits from Object by definition. If you look at the inheritance structure of any class, it will always end with Object on the top of the tree, as shown in Figure 6.3. **Türkçe:** Buna karşılık mevcut bir class'ı extend eden yeni bir class tanımladığınızda Java, declaration'a ayrıca `extends Object` eklemes. Bütün class'lar zaten `Object`'ten inheritance aldığı için child class da parent zinciri üzerinden `Object`'ten inheritance alır. Herhangi bir class'ın inheritance yapısını izlediğinizde ağacın tepesinde her zaman Şekil 6.3'teki gibi `Object` bulunur.

English: FIGURE 6.3 Java object inheritance **Türkçe:** ŞEKİL 6.3 Java object inheritance yapısı

English: `java.lang.Object ...` **Türkçe:** `java.lang.Object ...`

English: Mammal **Türkçe:** Mammal

English: Ox **Türkçe:** Ox

English: All objects inherit `java.lang.Object` **Türkçe:** Bütün object'ler `java.lang.Object`'ten inheritance alır

English: Primitive types such as `int` and `boolean` do not inherit from Object, since they are not classes. As you learned in Chapter 5, through autoboxing they can be assigned or passed as an instance of an associated wrapper class, which does inherit Object. **Türkçe:** `int` ve `boolean` gibi primitive type'lar class olmadığından `Object`'ten inheritance almaz. Chapter 5'te öğrendiğiniz autoboxing sayesinde bu değerler, `Object`'ten inheritance alan ilgili wrapper class'ın instance'ı biçiminde atanabilir veya argument olarak geçirilebilir.

Source page 0281

Creating Classes

Türkçe başlık: Class Oluşturma

English: Now that we've established how inheritance works in Java, we can use it to define and create complex class relationships. In this section, we review the basics for creating and working with classes.

Türkçe: Java'da inheritance'ın nasıl çalıştığını gördüğümüze göre artık karmaşık class ilişkileri tanımlayıp oluşturabiliriz. Bu kısımda class oluşturma ve class'larla çalışmanın temellerini gözden geçireceğiz.

Extending a Class

Türkçe başlık: Bir Class'ı Extend Etmek

English: Let's create two files in the same package, `Animal.java` and `Lion.java`. **Türkçe:** Aynı package içinde `Animal.java` ve `Lion.java` adlı iki dosya oluşturalım.

```
// Animal.java
public class Animal {
    private int age;
    protected String name;
    public int getAge() {
        return age;
    }
    public void setAge(int newAge) {
        age = newAge;
    }
}

// Lion.java
public class Lion extends Animal {
    protected void setProperties(int age, String n) {
        setAge(age);
        name = n;
    }
    public void roar() {
        System.out.print(name + ", age " + getAge() + ", says: Roar!");
    }
    public static void main(String[] args) {
        var lion = new Lion();
        lion.setProperties(3, "kion");
        lion.roar();
    }
}
```

Source page 0282

English: There's a lot going on here, we know! The age variable exists in the parent Animal class and is not directly accessible in the Lion child class. It is indirectly accessible via the setAge() method. The name variable is protected, so it is inherited in the Lion class and directly accessible. We create the Lion instance in the main() method and use setProperties() to set instance variables. Finally, we call the roar() method, which prints the following: kion, age 3, says: Roar! **Türkçe:** Burada birçok ayrıntı var. `age` variable'ı parent `Animal` class'ında bulunur ve child `Lion` class'ından doğrudan erişilemez; fakat `setAge()` method'u üzerinden dolayı olarak erişilebilir. `name` variable'ı `protected` olduğu için `Lion` tarafından inherit edilir ve doğrudan erişilebilir. `main()` içinde bir `Lion` instance'ı oluşturur, instance variable'larını `setProperties()` ile ayarlar ve son olarak `roar()` method'unu çağırırız. Çıktı şöyledir: `kion, age 3, says: Roar!`

English: Let's take a look at the members of the Lion class. The instance variable age is marked private and is not directly accessible from the subclass Lion. Therefore, the following would not compile: **Türkçe:** `Lion` class'ının member'larına bakalım. Instance variable `age`, `private` olduğu için `Lion` subclass'ından doğrudan erişilemez. Bu nedenle aşağıdaki kod derlenmez:

```
public class Lion extends Animal {
    public void roar() {
        System.out.print("Lions age: " + age); // DOES NOT COMPILE
    }
}
```

English: Remember when working with subclasses that private members are never inherited, and package members are only inherited if the two classes are in the same package. If you need a refresher on access modifiers, it may help to read Chapter 5 again. **Türkçe:** Subclass'larla çalışırken `private` member'ların hiçbir zaman doğrudan inherit edilmediğini; package-access member'ların ise yalnızca iki class aynı package içindeyse erişilebilir olduğunu unutmayın. Access modifier bilgilerinizi tazelemeniz gerekiyorsa Chapter 5'i yeniden okuyabilirsiniz.

Applying Class Access Modifiers

Türkçe başlık: Class Access Modifier'larını Uygulamak

English: Like variables and methods, you can apply access modifiers to classes. As you might remember from Chapter 1, a top-level class is one not defined inside another class. Also remember that a .java file can have at most one top-level class. **Türkçe:** Variable ve method'larda olduğu gibi class'lara da access modifier uygulanabilir. Chapter 1'den hatırlayacağınız üzere top-level class, başka bir class'ın içinde tanımlanmayan class'tır. Ayrıca bir .java dosyasında en fazla bir `public` top-level class bulunabileceğini unutmayın.

English: While you can only have one top-level class, you can have as many classes (in any order) with package access as you want. In fact, you don't even need to declare a public class! The following declares three classes, each with package access: **Türkçe:** Yalnızca bir `public` top-level class bulunabilir; ancak package access'e sahip istediğiniz sayıda class'ı istediğiniz sırada yazabilirsiniz. Hatta hiç `public` class bildirmek zorunda değilsiniz. Aşağıdaki üç class'ın da erişimi package'tir:

```
// Bear.java
class Bird {}
class Bear {}
class Fish {}
```

English: Trying to declare a top-level class with protected or private class will lead to a compiler error, though: **Türkçe:** Buna karşılık top-level bir class'ı `protected` veya `private` olarak bildirmek compiler error'a yol açar:

```
// ClownFish.java
protected class ClownFish{} // DOES NOT COMPILE
// BlueTang.java
private class BlueTang {} // DOES NOT COMPILE
```

Source page 0283

English: Does that mean a class can never be declared protected or private? Not exactly. In Chapter 7, we present nested types and show that when you define a class inside another, it can use any access modifier. **Türkçe:** Bu, bir class'ın hiçbir zaman `protected` veya `private` bildirilemeyeceği anlamına mı gelir? Hayır. Chapter 7'de nested type'ları tanıttık ve başka bir class'ın içinde tanımlanan class'ın herhangi bir access modifier'ı kullanabildiğini göreceğiz.

English: Accessing the this Reference **Türkçe:** `this` Reference'ına Erişmek

English: What happens when a method parameter has the same name as an existing instance variable? Let's take a look at an example. What do you think the following program prints? **Türkçe:** Bir method parameter, mevcut bir instance variable ile aynı adı taşıdığına ne olur? Örneğe bakalım: Sizce aşağıdaki program ne yazdırır?

```
public class Flamingo {
    private String color = null;
    public void setColor(String color) {
        color = color;
    }
    public static void main(String... unused) {
        var f = new Flamingo();
        f.setColor("PINK");
        System.out.print(f.color);
    }
}
```

English: If you said null, then you'd be correct. Java uses the most granular scope, so when it sees `color = color`, it thinks you are assigning the method parameter value to itself (not the instance variable). The assignment completes successfully within the method, but the value of the instance variable `color` is never modified and is null when printed in the `main()` method. **Türkçe:** `null` dediyeniz doğru. Java en dar scope'taki adı kullanır. Bu nedenle `color = color` gördüğünde, instance variable'a değil method parameter'a yine kendi değerinin atandığını kabul eder. Assignment method içinde geçerlidir; fakat instance variable `color` hiç değişmediği için `main()` içinde yazdırıldığında değer `null` olur.

English: The fix when you have a local variable with the same name as an instance variable is to use the `this` reference or keyword. The `this` reference refers to the current instance of the class and can be used to access any member of the class, including inherited members. It can be used in any instance method, constructor, or instance initializer block. It cannot be used when there is no implicit instance of the class, such as in a static method or static initializer block. We apply this to our previous method implementation as follows: **Türkçe:** Local variable ile instance variable aynı adı taşıyorsa çözüm `this` reference'ını/keyword'ünü kullanmaktır. `this`, class'ın o anki instance'ını gösterir ve inherit edilenler dahil class'ın bütün erişilebilir member'larına ulaşmak için kullanılabilir. Her instance method, constructor veya instance initializer block içinde kullanılabilir. Buna karşılık static method ya da static initializer block gibi örtük bir class instance'ının bulunmadığı yerlerde kullanılamaz. Önceki method'u şöyle düzeltiriz:

```
public void setColor(String color) {
    this.color = color; // Sets the instance variable with method parameter
}
```

English: The corrected code will now print PINK as expected. In many cases, the `this` reference is optional. If Java encounters a variable or method it cannot find, it will check the class hierarchy to see if it is available. **Türkçe:** Düzeltilmiş kod artık beklendiği gibi `PINK` yazdırır. Birçok durumda `this` reference'ı isteğe bağlıdır. Java bulunduğu scope'ta tanımlı olmayan bir variable veya method ile karşılaşırsa, erişilebilir olup olmadığını anlamak için class hierarchy'yi yukarı doğru kontrol eder.

English: Now let's look at some examples that aren't common but that you might see on the exam. **Türkçe:** Şimdi çok yaygın olmayan ama sınavda görebileceğiniz bazı örneklere bakalım.

```
1: public class Duck {  
2: private String color;  
3: private int height;
```

Source page 0284

```
4: private int length;  
5:  
6: public void setData(int length, int theHeight) {  
7: length = this.length; // Backwards--no good!  
8: height = theHeight; // Fine, because a different name  
9: this.color = "white"; // Fine, but this. reference not necessary  
10: }  
11:  
12: public static void main(String[] args) {  
13: Duck b = new Duck();  
14: b.setData(1,2);  
15: System.out.print(b.length + " " + b.height + " " + b.color);  
16: } }
```

English: This code compiles and prints the following: **Türkçe:** Bu kod derlenir ve aşağıdaki çıktıyı üretir:

```
0 2 white
```

English: This might not be what you expected, though. Line 7 is incorrect, and you should watch for it on the exam. The instance variable `length` starts out with a 0 value. That 0 is assigned to the method parameter `length`. The instance variable stays at 0. Line 8 is more straightforward. The parameter `theHeight` and instance variable `height` have different names. **Türkçe:** Bu, beklediğiniz sonuç olmayabilir. Line 7 yanlıştır ve sınavda bu kalıba dikkat etmelisiniz. Instance variable `length` başlangıçta 0 değerindedir; bu 0, method parameter `length`'e atanır ve instance variable 0 olarak kalır. Line 8 daha açıktır: Parameter `theHeight` ile instance variable `height` farklı adlara sahiptir.

English: Since there is no naming collision, this is not required. Finally, line 9 shows that a variable assignment is allowed to use the `this` reference even when there is no duplication of variable names.

Türkçe: Naming collision bulunmadığı için `this` gerekli değildir. Son olarak line 9, variable adları çakışmasa bile assignment'ta `this` reference'ının kullanılabileceğini gösterir.

English: Calling the super Reference **Türkçe:** `super` Reference'ını Kullanmak

English: In Java, a variable or method can be defined in both a parent class and a child class. This means the object instance actually holds two copies of the same variable with the same underlying name. When this happens, how do we reference the version in the parent class instead of the current class? Let's take a look at an example. **Türkçe:** Java'da aynı adlı bir variable veya method hem parent class'ta hem child class'ta tanımlanabilir. Böyle bir object instance'ı aynı temel ada sahip variable'ın iki ayrı kopyasını barındırır. Peki current class'taki sürüm yerine parent class'taki sürüme nasıl erişiriz? Bir örneğe bakalım.

```
// Reptile.java
1: public class Reptile {
2:     protected int speed = 10;
3: }
// Crocodile.java
1: public class Crocodile extends Reptile {
2:     protected int speed = 20;
3:     public int getSpeed() {
4:         return speed;
5:     }
6:     public static void main(String[] data) {
```

Source page 0285

```
7:     var croc = new Crocodile();
8:     System.out.println(croc.getSpeed()); // 20
9: } }
```

English: One of the most important things to remember about this code is that an instance of `Crocodile` stores two separate values for `speed`: one at the `Reptile` level and one at the `Crocodile` level. On line 4, Java first checks to see if there is a local variable or method parameter named `speed`. Since there is not, it then checks `this.speed`; and since it exists, the program prints 20. **Türkçe:** Bu kodda `Crocodile` instance'ı `speed` için iki ayrı değer saklar: biri `Reptile`, diğeri `Crocodile` düzeyindedir. Line 4'te Java önce `speed` adlı local variable veya method parameter arar. Bulamayınca `this.speed`'i kontrol eder; bu field bulunduğu için program 20 yazdırır.

English: Declaring a variable with the same name as an inherited variable is referred to as hiding a variable and is discussed later in this chapter. **Türkçe:** Inherit edilen bir variable ile aynı adı taşıyan yeni bir variable bildirmeye variable hiding denir. Konuyu bölümün ilerleyen kısmında ele alacağız.

English: But what if we want the program to print the value in the `Reptile` class? Within the `Crocodile` class, we can access the parent value of `speed`, instead, by using the `super` reference or keyword. The `super` reference is similar to the `this` reference, except that it excludes any members found in the current class. In other words, the member must be accessible via inheritance. **Türkçe:** Programın `Reptile` class'ındaki değeri yazdırmasını istersek `Crocodile` içinde `super` reference'ını/keyword'ünü kullanarak parent'taki `speed` değerine erişebiliriz. `super`, `this` e benzer; ancak current class'ta tanımlanan member'ları dışarıda bırakır. Başka bir deyişle member'ın inheritance yoluyla erişilebilir olması gerekir.

```
3:     public int getSpeed() {
4:         return super.speed; // Causes the program to now print 10
5:     }
```

English: Let's see if you've gotten the hang of this and `super`. What does the following program output?

Türkçe: `this` ile `super` konusunu kavrayıp kavramadığınızı görelim. Aşağıdaki programın çıktısı nedir?

```
1: class Insect {
2: protected int numberOfLegs = 4;
3: String label = "buggy";
4: }
5:
6: public class Beetle extends Insect {
7: protected int numberOfLegs = 6;
8: short age = 3;
9: public void printData() {
10: System.out.println(this.label);
11: System.out.println(super.label);
12: System.out.println(this.age);
13: System.out.println(super.age);
14: System.out.println(numberOfLegs);
15: }
16: public static void main(String []n) {
17: new Beetle().printData();
18: }
19: }
```

Source page 0286

English: That was a trick question— this program code would not compile! Let's review each line of the printData() method. Since label is defined in the parent class, it is accessible via both this and super references. For this reason, lines 10 and 11 compile and would both print buggy if the class compiled. On the other hand, the variable age is defined only in the current class, making it accessible via this but not super. For this reason, line 12 compiles (and would print 3), but line 13 does not. Remember, while this includes current and inherited members, super only includes inherited members. **Türkçe:** Bu bir tuzak soruydu: Program derlenmez. printData() içindeki satırları inceleyelim. label parent class'ta tanımlandığından hem this hem super üzerinden erişilebilir; bu yüzden line 10 ve 11 derlenir ve class derlenebilse ikisi de buggy yazdırırdı. age ise yalnızca current class'ta tanımlıdır; this ile erişilebilir, super ile erişilemez. Bu nedenle line 12 derlenip 3 yazdıracakken line 13 derlenmez. this current ve inherited member'ları; super ise yalnızca inherited member'ları kapsar.

English: Last but not least, what would line 14 print if line 13 was commented out? Even though both numberOfLegs variables are accessible in Beetle, Java checks outward, starting with the narrowest scope. For this reason, the value of numberOfLegs in the Beetle class is used, and 6 is printed. In this example, this.numberOfLegs and super.numberOfLegs refer to different variables with distinct values.

Türkçe: Son olarak line 13 comment yapılırsa line 14 ne yazdırır? Beetle içinden iki numberOfLegs variable'ına da erişilebilse de Java en dar scope'tan başlayıp dışarı doğru arar. Bu nedenle Beetle class'ındaki değer kullanılır ve 6 yazdırılır. Bu örnekte this.numberOfLegs ile super.numberOfLegs, değerleri farklı iki ayrı variable'ı gösterir.

English: Since this includes inherited members, you often only use super when you have a naming conflict via inheritance. For example, you have a method or variable defined in the current class that matches a method or variable in a parent class. This commonly comes up in method overriding and variable hiding, which are discussed later in this chapter. **Türkçe:** this, inherited member'ları da kapsadığı için super çoğunlukla inheritance kaynaklı bir naming conflict olduğunda gerekir. Örneğin current class'taki bir method veya variable, parent class'takiyle aynı adı taşıyabilir. Bu durum özellikle ileride ele alınacak method overriding ve variable hiding konularında görülür.

English: Phew, that was a lot! Using this and super can take a little getting used to. Since we use them often in upcoming sections, make sure you understand the last example really well before moving forward.

Türkçe: Epey ayrıntı vardı! `this` ve `super` kullanımına alışmak biraz zaman alabilir. Sonraki kısımlarda ikisini de sık kullanacağımız için ilerlemeden önce son örneği iyice anladığınızdan emin olun.

Declaring Constructors

Türkçe başlık: Constructor Declaration'ları

English: As you learned in Chapter 1, a constructor is a special method that matches the name of the class and has no return type. It is called when a new instance of the class is created. For the exam, you'll need to know a lot of rules about constructors. In this section, we show how to create a constructor. Then, we look at default constructors, overloading constructors, calling parent constructors, final fields, and the order of initialization in a class.

Türkçe: Chapter 1'de öğrendiğiniz gibi constructor, class ile aynı adı taşıyan ve return type'ı bulunmayan özel bir method'dur. Class'ın yeni bir instance'ı oluşturulduğunda çağrılır. Sınavda constructor'larla ilgili birçok kuralı bilmeniz gerekir. Önce constructor oluşturmayı; ardından default constructor'ı, constructor overloading'i, parent constructor çağrılarını, `final` field'ları ve class içindeki initialization order'ı inceleyeceğiz.

Creating a Constructor

Türkçe başlık: Constructor Oluşturmak

English: Let's start with a simple constructor: **Türkçe:** Basit bir constructor'la başlayalım:

```
public class Bunny {
    public Bunny() {
        System.out.print("hop");
    }
}
```

English: The name of the constructor, `Bunny`, matches the name of the class, `Bunny`, and there is no return type, not even `void`. That makes this a constructor. Can you tell why these two are not valid constructors for the `Bunny` class? **Türkçe:** Constructor'ın adı `Bunny`, class'ın adı olan `Bunny` ile aynıdır ve `void` dahil hiçbir return type yoktur; bu nedenle bu declaration bir constructor'dır. Aşağıdaki ikisinin neden `Bunny` için geçerli constructor olmadığını söyleyebilir misiniz?

```
public class Bunny {
```

Source page 0287

```
public bunny() {} public void Bunny() {}
// DOES NOT COMPILE
}
```

English: The first one doesn't match the class name because Java is case-sensitive. Since it doesn't match, Java knows it can't be a constructor and is supposed to be a regular method. However, it is missing the return type and doesn't compile. The second method is a perfectly good method but is not a constructor because it has a return type. **Türkçe:** Java case-sensitive olduğundan ilk declaration'ın adı class adıyla eşleşmez. Java bunun constructor olamayacağını, regular method olması gerektiğini anlar; fakat return type bulunmadığı için kod derlenmez. İkinci declaration geçerli bir method'dur, ancak return type'i olduğu için constructor değildir.

English: Like method parameters, constructor parameters can be any valid class, array, or primitive type, including generics, but may not include var. For example, the following does not compile: **Türkçe:** Method parameter'larında olduğu gibi constructor parameter'ları da generics dahil geçerli bir class, array veya primitive type olabilir; ancak `var` kullanamaz. Örneğin aşağıdaki kod derlenmez:

```
public class Bonobo {
    public Bonobo(var food) { // DOES NOT COMPILE
    }
}
```

English: A class can have multiple constructors, as long as each constructor has a unique constructor signature. In this case, that means the constructor parameters must be distinct. Like methods with the same name but different signatures, declaring multiple constructors with different signatures is referred to as constructor overloading. The following Turtle class has four distinct overloaded constructors: **Türkçe:** Her birinin constructor signature'ı benzersiz olmak koşuluyla bir class birden fazla constructor içerebilir. Burada parameter list'lerin farklı olması gerekir. Aynı adlı fakat farklı signature'lı method'lara benzer biçimde, farklı signature'lara sahip birden çok constructor bildirmeye constructor overloading denir. Aşağıdaki `Turtle` class'ında birbirinden farklı dört overloaded constructor vardır:

```
public class Turtle {
    private String name;
    public Turtle() {
        name = "John Doe";
    }
    public Turtle(int age) {}
    public Turtle(long age) {}
    public Turtle(String newName, String... favoriteFoods) {
        name = newName;
    }
}
```

English: Constructors are used when creating a new object. This process is called instantiation because it creates a new instance of the class. A constructor is called when we write new followed by the name of the class we want to instantiate. Here's an example: **Türkçe:** Constructor'lar yeni object oluştururken kullanılır. Class'ın yeni bir instance'ını oluşturduğu için bu süreçte instantiation denir. Instantiate etmek istediğimiz class'ın adından önce `new` yazdığımızda constructor çağrılır. Örneğin:

```
new Turtle(15)
```

English: When Java sees the new keyword, it allocates memory for the new object. It then looks for a constructor with a matching signature and calls it. **Türkçe:** Java `new` keyword'ünü gördüğünde yeni object için memory ayırır; ardından eşleşen signature'a sahip constructor'ı bulup çağırır.

The Default Constructor

Türkçe başlık: Varsayılan Constructor

English: Every class in Java has a constructor, whether you code one or not. If you don't include any constructors in the class, Java will create one for you without any parameters. **Türkçe:** Java'daki her class'ın, siz yazsanız da yazmasanız da bir constructor'ı vardır. Class içinde hiçbir constructor bildirmezseniz Java sizin için parameter almayan bir constructor oluşturur.

Source page 0288

English: This Java-created constructor is called the default constructor and is added any time a class is declared without any constructors. We often refer to it as the default no-argument constructor, for clarity. Here's an example: **Türkçe:** Java'nın oluşturduğu bu constructor'a default constructor denir ve yalnızca class'ta hiçbir constructor bildirilmediğinde eklenir. Açık olması için çoğu zaman default no-argument constructor ifadesini kullanırız. Örneğin:

```
public class Rabbit {  
    public static void main(String[] args) {  
        new Rabbit(); // Calls the default constructor  
    }  
}
```

English: In the Rabbit class, Java sees that no constructor was coded and creates one. The previous class is equivalent to the following, in which the default constructor is provided and therefore not inserted by the compiler: **Türkçe:** Java, `Rabbit` class'ında constructor yazılmadığını görür ve bir tane üretir. Önceki class aşağıdakiyle eşdeğerdir; ancak burada constructor kullanıcı tarafından yazıldığı için compiler ayrıca default constructor eklemeyiz:

```
public class Rabbit {  
    public Rabbit() {}  
    public static void main(String[] args) {  
        new Rabbit(); // Calls the user-defined constructor  
    }  
}
```

English: The default constructor has an empty parameter list and an empty body. It is fine for you to type this in yourself. However, since it doesn't do anything, Java is happy to generate it for you and save you some typing. **Türkçe:** Default constructor'ın parameter list'i ve body'si boştur. Aynı declaration'ı kendiniz de yazabilirsiniz; ancak hiçbir işlem yapmadığı için Java bunu otomatik üreterek sizi gereksiz yazımdan kurtarır.

English: We keep saying generated. This happens during the compile step. If you look at the file with the .java extension, the constructor will still be missing. It only makes an appearance in the compiled file with the .class extension. **Türkçe:** “Üretilir” derken bunun compile aşamasında gerçekleştiğini kastediyoruz. .java source file'a baktığınızda constructor hâlâ görünmez; yalnızca derlenmiş .class file içinde yer alır.

English: For the exam, one of the most important rules you need to know is that the compiler only inserts the default constructor when no constructors are defined. Which of these classes do you think has a default constructor? **Türkçe:** Sınav için en önemli kurallardan biri şudur: Compiler, default constructor'ı yalnızca hiç constructor tanımlanmamışsa ekler. Sizce aşağıdaki class'lardan hangisine default constructor eklenir?

```
public class Rabbit1 {}  
public class Rabbit2 {  
    public Rabbit2() {}  
}  
public class Rabbit3 {  
    public Rabbit3(boolean b) {}  
}  
public class Rabbit4 {  
    private Rabbit4() {}  
}
```

Source page 0289

English: Only Rabbit1 gets a default no-argument constructor. It doesn't have a constructor coded, so Java generates a default no-argument constructor. Rabbit2 and Rabbit3 both have public constructors already. Rabbit4 has a private constructor. Since these three classes have a constructor defined, the default no-argument constructor is not inserted for you. **Türkçe:** Yalnızca `Rabbit1` default no-argument constructor alır; çünkü içinde yazılmış bir constructor yoktur. `Rabbit2` ve `Rabbit3` zaten `public`, `Rabbit4` ise `private` bir constructor bildirir. Bu üç class'ta constructor tanımlandığından compiler default no-argument constructor eklemeyiz.

English: Let's take a quick look at how to call these constructors: **Türkçe:** Bu constructor'ların nasıl çağırıldığına kısaca bakalım:

```
1: public class RabbitsMultiply {  
2:     public static void main(String[] args) {  
3:         var r1 = new Rabbit1();  
4:         var r2 = new Rabbit2();  
5:         var r3 = new Rabbit3(true);  
6:         var r4 = new Rabbit4(); // DOES NOT COMPILE  
7:     } }
```

English: Line 3 calls the generated default no-argument constructor. Lines 4 and 5 call the user-provided constructors. Line 6 does not compile. Rabbit4 made the constructor private so that other classes could not call it. **Türkçe:** Line 3 compiler'ın ürettiği default no-argument constructor'ı çağırır. Line 4 ve 5 kullanıcı tarafından yazılmış constructor'ları çağırır. Line 6 derlenmez; çünkü `Rabbit4`, başka class'ların çağırmasını önlemek için constructor'ını `private` yapmıştır.

English: Having only private constructors in a class tells the compiler not to provide a default no-argument constructor. It also prevents other classes from instantiating the class. This is useful when a class has only static methods or the developer wants to have full control of all calls to create new instances of the class. **Türkçe:** Bir class'ta yalnızca `private` constructor bulunması, compiler'ın default no-argument constructor eklemesini önler; ayrıca başka class'ların bu class'ı instantiate etmesine izin vermez. Bu yaklaşım, class yalnızca static method'lar içerdiğinde veya geliştirici yeni instance oluşturma çağrılarını bütünüyle denetlemek istediğinde yararlıdır.

English: Calling Overloaded Constructors with `this()` Have the basics about creating and referencing constructors? Good, because things are about to get a bit more complicated. Since a class can contain multiple overloaded constructors, these constructors can actually call one another. Let's start with a simple class containing two overloaded constructors: **Türkçe:** Overloaded Constructor'ları `this()` ile Çağırarak — Constructor oluşturma ve constructor'a başvurma temelleri hazırsa biraz daha karmaşık kurallara geçebiliriz. Bir class birden fazla overloaded constructor içerebilir ve bu constructor'lar birbirini çağırabilir. İki overloaded constructor içeren basit bir class ile başlayalım:

```
public class Hamster {
    private String color;
    private int weight;
    public Hamster(int weight, String color) { // First constructor
        this.weight = weight;
        this.color = color;
    }
    public Hamster(int weight) { // Second constructor
        this.weight = weight;
        color = "brown";
    }
}
```

Source page 0290

English: One of the constructors takes a single int parameter. The other takes an int and a String. These parameter lists are different, so the constructors are successfully overloaded. **Türkçe:** Constructor'lardan biri tek bir `int`, diğeri bir `int` ve bir `String` parameter alır. Parameter list'ler farklı olduğundan constructor'lar geçerli biçimde overload edilmiştir.

English: There is a bit of duplication, as `this.weight` is assigned the same way in both constructors. In programming, even a bit of duplication tends to turn into a lot of duplication as we keep adding “just one more thing.” For example, imagine that we have five variables being set like `this.weight`, rather than just one. What we really want is for the first constructor to call the second constructor with two parameters. So, how can you have a constructor call another constructor? You might be tempted to rewrite the first constructor as the following: **Türkçe:** `this.weight` iki constructor'da da aynı şekilde atandığı için kod tekrarı vardır. Programlamada küçük bir tekrar, “bir şey daha” eklendikçe kolayca büyür. Tek bir variable yerine `this.weight` gibi ayarlanan beş variable olduğunu düşünün. İstedikimiz, tek parameter alan constructor'ın iki parameter alan constructor'ı çağırmasıdır. Bunun için ilk akla gelen, constructor'ı şöyle yazmak olabilir:

```
public Hamster(int weight) { // Second constructor
    Hamster(weight, "brown"); // DOES NOT COMPILE
}
```

English: This will not work. Constructors can be called only by writing `new` before the name of the constructor. They are not like normal methods that you can just call. What happens if we stick `new` before the constructor name? **Türkçe:** Bu çalışmaz. Constructor, normal method gibi doğrudan çağrılmaz; constructor adının önüne `new` yazılarak çağrılır. Peki başına `new` eklersek ne olur?

```
public Hamster(int weight) { // Second constructor
    new Hamster(weight, "brown"); // Compiles, but creates an extra object
}
```

English: This attempt does compile. It doesn't do what we want, though. When this constructor is called, it creates a new object with the default weight and color. It then constructs a different object with the desired weight and color. In this manner, we end up with two objects, one of which is discarded after it is created. That's not what we want. We want weight and color set on the object we are trying to instantiate in the first place. **Türkçe:** Bu sürüm derlenir, fakat istediğimizi yapmaz. İlk constructor çağrısı default `weight` ve `color` değerlerine sahip bir object oluştururken, `new Hamster(...)` istenen değerlerle ikinci ve farklı bir object oluşturur. Sonuçta iki object vardır ve ikincisi oluşturulduktan sonra kullanılmadan atılır. Oysa asıl instantiate etmekte olduğumuz object'in `weight` ve `color` değerlerini ayarlamak istiyoruz.

English: Java provides a solution: `this()`— **Türkçe:** Java bir çözüm sunar: `this()`—

English: yes, the same keyword we used to refer to instance members, but with parentheses. When `this()` is used with parentheses, Java calls another constructor on the same instance of the class. **Türkçe:** Evet, instance member'lara erişirken kullandığımız keyword'ün parantezli biçimi. `this()` kullanıldığında Java, aynı class instance'ındaki başka bir constructor'ı çağırır.

```
public Hamster(int weight) { // Second constructor
    this(weight, "brown");
}
```

English: Success! Now Java calls the constructor that takes two parameters, with weight and color set as expected. **Türkçe:** Artık Java iki parameter alan constructor'ı çağırır ve `weight` ile `color` beklediği gibi ayarlanır.

this vs. this()

Türkçe başlık: `this` ile `this()` karşılaştırması

English: Despite using the same keyword, `this` and `this()` are very different. The first, `this`, refers to an instance of the class, while the second, `this()`, refers to a constructor call within the class. The exam may try to trick you by using both together, so make sure you know which one to use and why. **Türkçe:** Aynı keyword'ü kullansalar da `this` ve `this()` farklıdır. `this`, current class instance'ını; `this()` ise aynı class içindeki bir constructor çağrısını gösterir. Sınav ikisini birlikte kullanarak tuzak kurabilir; hangisinin nerede ve neden kullanıldığını bilmelisiniz.

Source page 0291

English: Calling `this()` has one special rule you need to know. If you choose to call it, the `this()` call must be the first statement in the constructor. The side effect of this is that there can be only one call to `this()` in any constructor. **Türkçe:** `this()` çağrısının özel bir kuralı vardır: Kullanılacaksa constructor içindeki first statement olmak zorundadır. Bunun sonucu olarak bir constructor içinde yalnızca bir `this()` çağrısı bulunabilir.

```
3: public Hamster(int weight) {
4:     System.out.println("chew");
5:     // Set weight and default color
6:     this(weight, "brown"); // DOES NOT COMPILE
7: }
```

English: Even though a print statement on line 4 doesn't change any variables, it is still a Java statement and is not allowed to be inserted before the call to `this()`. The comment on line 5 is just fine. Comments aren't considered statements and are allowed anywhere. **Türkçe:** Line 4'teki print statement hiçbir variable'ı değiştirmese de yine bir Java statement'ıdır ve `this()` çağrısından önce gelemez. Line 5'teki comment sorun değildir; comment'ler statement sayılmaz ve her yerde bulunabilir.

English: There's one last rule for overloaded constructors that you should be aware of. Consider the following definition of the Gopher class: **Türkçe:** Overloaded constructor'larla ilgili son bir kural daha vardır. Aşağıdaki `Gopher` class'ını inceleyin:

```
public class Gopher {
    public Gopher(int dugHoles) {
        this(5); // DOES NOT COMPILE
    }
}
```

English: The compiler is capable of detecting that this constructor is calling itself infinitely. This is often referred to as a cycle and is similar to the infinite loops that we discussed in Chapter 3, "Making Decisions." Since the code can never terminate, the compiler stops and reports this as an error. Likewise, this also does not compile: **Türkçe:** Compiler, bu constructor'ın kendisini sonsuza kadar çağıracağını saptayabilir. Buna cycle denir ve Chapter 3, "Making Decisions"ta ele alınan infinite loop'lara benzer. Çağrı zinciri hiçbir zaman sonlanamayacağı için compiler bunu error olarak bildirir. Benzer biçimde aşağıdaki kod da derlenmez:

```
public class Gopher {  
    public Gopher() {  
        this(5); // DOES NOT COMPILE  
    }  
    public Gopher(int dugHoles) {  
        this(); // DOES NOT COMPILE  
    }  
}
```

English: In this example, the constructors call each other, and the process continues infinitely. Since the compiler can detect this, it reports an error. **Türkçe:** Bu örnekte iki constructor birbirini çağırır ve zincir sonsuza kadar sürer. Compiler cycle'ı saptayabildiği için error verir.

English: Here we summarize the rules you should know about constructors that we covered in this section. Study them well! **Türkçe:** Bu kısımda ele alınan constructor kurallarını şöyle özetleyebiliriz; bunları iyi çalışın:

English: ■ ■ A class can contain many overloaded constructors, provided the signature for each is distinct. **Türkçe:** ■ ■ Her birinin signature'ı farklı olmak koşuluyla bir class birçok overloaded constructor içerebilir.

English: ■ ■ ■ ■ ■ The compiler inserts a default no-argument constructor if no constructors are declared. **Türkçe:** ■ ■ ■ ■ ■ Hiç constructor bildirilmezse compiler default no-argument constructor ekler.

English: If a constructor calls `this()`, then it must be the first line of the constructor. **Türkçe:** Bir constructor `this()` çağırıyorsa bu çağrı constructor'ın first statement'ı olmalıdır.

English: Java does not allow cyclic constructor calls. **Türkçe:** Java cyclic constructor call'lara izin vermez.

Source page 0292

English: Calling Parent Constructors with `super()` **Türkçe:** Parent Constructor'ları `super()` ile Çağırarak

English: Congratulations: you're well on your way to becoming an expert in using constructors! **Türkçe:** Tebrikler: constructor'ları kullanma konusunda uzman olma yolunda ilerliyorsunuz!

English: There's one more set of rules we need to cover, though, for calling constructors in the parent class. After all, how do instance members of the parent class get initialized? **Türkçe:** Bununla birlikte parent class'taki constructor'ları çağırmaya ilişkin bir kural grubu daha vardır. Parent class'ın instance member'ları başka nasıl initialize edilebilir?

English: The first statement of every constructor is a call to a parent constructor using `super()` or another constructor in the class using `this()`. Read the previous sentence twice to make sure you remember it. It's really important! **Türkçe:** Her constructor'ın first statement'ı ya `super()` ile parent constructor'a ya da `this()` ile aynı class'taki başka bir constructor'a yapılan çağrıdır. Bu kural çok önemlidir; kalıcı olduğundan emin olun.

English: For simplicity in this section, we often refer to `super()` and `this()` to refer to any parent or overloaded constructor call, even those that take arguments. **Türkçe:** Bu kısımda anlatımı sade tutmak için argument alan çağrılar dahil bütün parent ve overloaded constructor çağrılarını çoğu zaman genel olarak `super()` ve `this()` diye adlandıracakız.

English: Let's take a look at the `Animal` class and its subclass `Zebra` and see how their constructors can be properly written to call one another: **Türkçe:** `Animal` class'ı ile subclass'ı `Zebra`'ya bakalım ve constructor'ların birbirini doğru biçimde nasıl çağırdığını görelim:

```
public class Animal {
    private int age;
    public Animal(int age) {
        super(); // Refers to constructor in java.lang.Object
        this.age = age;
    }
}

public class Zebra extends Animal {
    public Zebra(int age) {
        super(age); // Refers to constructor in Animal
    }
    public Zebra() {
        this(4); // Refers to constructor in Zebra with int argument
    }
}
```

English: In the `Animal` class, the first statement of the constructor is a call to the parent constructor defined in `java.lang.Object`, which takes no arguments. In the second class, `Zebra`, the first statement of the first constructor is a call to `Animal`'s constructor, which takes a single argument. The `Zebra` class also includes a second no-argument constructor that doesn't call `super()` but instead calls the other constructor within the `Zebra` class using `this(4)`. **Türkçe:** `Animal` class'ında constructor'ın first statement'ı, `java.lang.Object` içinde tanımlanan no-argument parent constructor'a çağrıdır. `Zebra` class'ındaki ilk constructor'ın first statement'ı ise tek argument alan `Animal` constructor'ını çağırır. `Zebra` ayrıca `super()` yerine `this(4)` ile aynı class'taki diğer constructor'ı çağıran ikinci bir no-argument constructor içerir.

Source page 0293

super vs. super()

Türkçe başlık: `super` ile `super()` karşılaştırması

English: Like this and this(), super and super() are unrelated in Java. The first, super, is used to reference members of the parent class, while the second, super(), calls a parent constructor. Anytime you see the keyword super on the exam, make sure it is being used properly. **Türkçe:** this ile this() gibi, super ile super() da Java'da farklı amaçlara sahiptir. super, parent class member'larına erişir; super() ise parent constructor'ı çağırır. Sınavda super keyword'ünü gördüğünüzde hangi biçimin kullanıldığını kontrol edin.

English: Like calling this(), calling super() can only be used as the first statement of the constructor.

Türkçe: this() gibi super() da yalnızca constructor'ın first statement'ı olarak kullanılabilir.

English: For example, the following two class definitions will not compile: **Türkçe:** Örneğin aşağıdaki iki class tanımı derlenmeyecektir:

```
public class Zoo {
    public Zoo() {
        System.out.println("Zoo created");
        super(); // DOES NOT COMPILE
    }
}

public class Zoo {
    public Zoo() {
        super();
        System.out.println("Zoo created");
        super(); // DOES NOT COMPILE
    }
}
```

English: The first class will not compile because the call to the parent constructor must be the first statement of the constructor. In the second code snippet, super() is the first statement of the constructor, but it is also used as the third statement. Since super() can only be called once as the first statement of the constructor, the code will not compile. **Türkçe:** İlk class derlenmez; çünkü parent constructor çağrısı constructor'ın first statement'ı olmalıdır. İkinci snippet'te super() first statement'tır fakat üçüncü statement olarak bir kez daha kullanılmıştır. super() yalnızca bir defa ve first statement olarak çağrılabilirdiğinden bu kod da derlenmez.

English: If the parent class has more than one constructor, the child class may use any valid parent constructor in its definition, as shown in the following example: **Türkçe:** Parent class'ta birden fazla constructor varsa child class, aşağıdaki örnekte olduğu gibi bunlardan geçerli olan herhangi birini çağırabilir:

```
public class Animal {
    private int age;
    private String name;
    public Animal(int age, String name) {
        super();
        this.age = age;
    }
}
```

Source page 0294

```
this.name = name;
}
public Animal(int age) {
    super();
    this.age = age;
    this.name = null;
}
}
public class Gorilla extends Animal {
    public Gorilla(int age) {
        super(age, "Gorilla"); // Calls the first Animal constructor
    }
    public Gorilla() {
        super(5); // Calls the second Animal constructor
    }
}
```

English: In this example, the first child constructor takes one argument, age, and calls the parent constructor, which takes two arguments, age and name. The second child constructor takes no arguments, and it calls the parent constructor, which takes one argument, age. In this example, notice that the child constructors are not required to call matching parent constructors. Any valid parent constructor is acceptable as long as the appropriate input parameters to the parent constructor are provided. **Türkçe:** Bu örnekte ilk child constructor bir argument (age) alır ve iki argument (age, name) alan parent constructor'ı çağırır. İkinci child constructor argument almaz ve tek argument (age) alan parent constructor'ı çağırır. Child constructor'ın parameter list'iyle parent constructor'ının eşleşmesi gerekmez; parent constructor için gerekli input parameter'lar sağlandığı sürece geçerli herhangi bir parent constructor çağırılabilir.

Understanding Compiler Enhancements

Türkçe başlık: Compiler'ın Eklediği Kodları Anlamak

English: Wait a second: we said the first line of every constructor is a call to either this() or super(), but we've been creating classes and constructors throughout this book, and we've rarely done either. How did these classes compile? **Türkçe:** Her constructor'ın ilk satırında this() veya super() çağırısı bulunduğunu söyledik; oysa kitap boyunca birçok class ve constructor yazdık ve bu çağrılar nadiren açıkça kullandık. Peki bu class'lar nasıl derlendi?

English: The answer is that the Java compiler automatically inserts a call to the no-argument constructor super() if you do not explicitly call this() or super() as the first line of a constructor. **Türkçe:** Çünkü constructor'ın ilk satırında açıkça this() veya super() çağırmazsanız Java compiler, no-argument parent constructor'a super() çağırısını otomatik ekler.

English: For example, the following three class and constructor definitions are equivalent, because the compiler will automatically convert them all to the last example: **Türkçe:** Bu nedenle aşağıdaki üç class/constructor tanımı eşdeğerdir; compiler hepsini son biçime dönüştürür:

```
public class Donkey {}  
public class Donkey {  
    public Donkey() {}  
}
```

Source page 0295

```
public class Donkey {  
    public Donkey() {  
        super();  
    }  
}
```

English: Make sure you understand the differences between these three Donkey class definitions and why Java will automatically convert them all to the last definition. While reading the next section, keep in mind the process the Java compiler performs. **Türkçe:** Üç *Donkey* declaration'ı arasındaki farkı ve Java'nın neden hepsini son biçime dönüştürdüğünü anladığınızdan emin olun. Sonraki kısmı okurken compiler'ın bu eklemesini aklınızda tutun.

Default Constructor Tips and Tricks

Türkçe başlık: Varsayılan Constructor İpuçları ve Püf Noktaları

English: We've presented a lot of rules so far, and you might have noticed something. Let's say we have a class that doesn't include a no-argument constructor. What happens if we define a subclass with no constructors, or a subclass with a constructor that doesn't include a `super()` reference? **Türkçe:** Şimdiye kadar birçok kural gördük. No-argument constructor içermeyen bir class düşünün. Constructor bildirmeyen bir subclass veya constructor'ında `super()` çağırısı bulunmayan bir subclass tanımlarsak ne olur?

```
public class Mammal {  
    public Mammal(int age) {}  
}  
public class Seal extends Mammal {} // DOES NOT COMPILE  
public class Elephant extends Mammal {  
    public Elephant() {} // DOES NOT COMPILE  
}
```

English: The answer is that neither subclass compiles. Since Mammal defines a constructor, the compiler does not insert a no-argument constructor. The compiler will insert a default no-argument constructor into Seal, though, but it will be a simple implementation that just calls a nonexistent parent default constructor. **Türkçe:** İki subclass da derlenmez. *Mammal* bir constructor tanımladığı için compiler ona no-argument constructor eklemeyiz. Compiler *Seal* class'ına default no-argument constructor ekler; fakat bu constructor, var olmayan no-argument parent constructor'ı çağırır.

```
public class Seal extends Mammal {  
    public Seal() {  
        super(); // DOES NOT COMPILE  
    }  
}
```

English: Likewise, Elephant will not compile for similar reasons. The compiler doesn't see a call to `super()` or `this()` as the first line of the constructor so it inserts a call to a nonexistent no-argument `super()` automatically. **Türkçe:** `Elephant` da aynı nedenle derlenmez. Compiler, constructor'ın first statement'ında `super()` veya `this()` görmediği için var olmayan no-argument parent constructor'a otomatik olarak `super()` çağırısı ekler.

```
public class Elephant extends Mammal {
    public Elephant() {
        super(); // DOES NOT COMPILE
    }
}
```

Source page 0296

English: In these cases, the compiler will not help, and you must create at least one constructor in your child class that explicitly calls a parent constructor via the `super()` command. **Türkçe:** Bu durumda child class'ta en az bir constructor yazmalı ve `super()` ile mevcut parent constructor'lardan birini açıkça çağırmalısınız.

```
public class Seal extends Mammal {
    public Seal() {
        super(6); // Explicit call to parent constructor
    }
}

public class Elephant extends Mammal {
    public Elephant() {
        super(4); // Explicit call to parent constructor
    }
}
```

English: Subclasses may include no-argument constructors even if their parent classes do not. For example, the following compiles because Elephant includes a no-argument constructor. **Türkçe:** Parent class'ta no-argument constructor bulunmasa bile subclass'ta bulunabilir. Örneğin `Elephant` no-argument constructor içerdiği için aşağıdaki kod derlenir:

```
public class AfricanElephant extends Elephant {}
```

English: It's a lot to take in, we know. For the exam, you should be able to spot right away why classes such as our first Seal and Elephant implementations did not compile. **Türkçe:** Kural sayısı fazla olsa da sınavda ilk `Seal` ve `Elephant` uygulamalarının neden derlenmediğini hızla görebilmelisiniz.

`super()` Always Refers to the Most Direct Parent

Türkçe başlık: `super()` Her Zaman En Yakın Doğrudan Parent'a Referans Verir

English: A class may have multiple ancestors via inheritance. In our previous example, `AfricanElephant` is a subclass of `Elephant`, which in turn is a subclass of `Mammal`. For constructors, though, `super()` always refers to the most direct parent. In this example, calling `super()` inside the `AfricanElephant` class always refers to the `Elephant` class and never to the `Mammal` class. **Türkçe:** Bir class'ın inheritance zincirinde birden fazla ancestor bulunabilir. Önceki örnekte `AfricanElephant`, `Elephant`'ın; `Elephant` da `Mammal`'ın subclass'ıdır. Buna rağmen constructor bağlamında `super()` her zaman direct parent'ı gösterir. Dolayısıyla `AfricanElephant` içindeki `super()` çağrısı `Elephant`'a gider; hiçbir zaman doğrudan `Mammal`'ı çağırılmaz.

English: We conclude this section by adding three constructor rules to your skill set: **Türkçe:** Bu kısmı üç constructor kuralıyla tamamlayalım:

English: ■ ■ The first line of every constructor is a call to a parent constructor using `super()` or an overloaded constructor using `this()`. **Türkçe:** ■ ■ Her constructor'ın first statement'ı `super()` ile parent constructor'a veya `this()` ile overloaded constructor'a yapılan çağrıdır.

English: ■ ■ If the constructor does not contain a `this()` or `super()` reference, then the compiler automatically inserts `super()` with no arguments as the first line of the constructor. **Türkçe:** ■ ■ Constructor açık bir `this()` veya `super()` çağrısı içermiyorsa compiler, first statement olarak no-argument `super()` çağrısı ekler.

English: ■ ■ If a constructor calls `super()`, then it must be the first line of the constructor. **Türkçe:** ■ ■ Constructor `super()` çağırıyorsa çağrı first statement olmalıdır.

English: Congratulations: you've learned everything we can teach you about declaring constructors. Next, we move on to initialization and discuss how to use constructors. **Türkçe:** Constructor declaration'larıyla ilgili kuralları tamamladık. Sırada initialization ve constructor'ların bu süreçteki kullanımı var.

Source page 0297

Initializing Objects

Türkçe başlık: Object'leri Initialize Etmek

English: In Chapter 1, we covered order of initialization, albeit in a very simplistic manner. Order of initialization refers to how members of a class are assigned values. They can be given default values, like 0 for an int, or require explicit values, such as for final variables. In this section, we go into much more detail about how order of initialization works and how to spot errors on the exam. **Türkçe:** Chapter 1'de initialization order'ı temel düzeyde ele aldık. Initialization order, class member'larına değerlerin hangi sırayla atandığını belirtir. Bir member `int` için 0 gibi default value alabilir veya `final` variable'larda olduğu gibi explicit value gerektirebilir. Bu kısımda initialization order'ın işleyişini ve sınavda ilgili hataların nasıl saptanacağını ayrıntılı inceleyeceğiz.

Initializing Classes

Türkçe başlık: Class'ları Initialize Etmek

English: We begin our discussion of order of initialization with class initialization. First, we initialize the class, which involves invoking all static members in the class hierarchy, starting with the highest superclass and working downward. This is sometimes referred to as loading the class. **Türkçe:** Initialization order konusuna class initialization ile başlıyoruz. Önce class initialize edilir: Class hierarchy'deki bütün static member'lar en üst superclass'tan başlanarak aşağı doğru çalıştırılır. Buna bazen class loading de denir.

English: The Java Virtual Machine (JVM) controls when the class is initialized, although you can assume the class is loaded before it is used. The class may be initialized when the program first starts, when a static member of the class is referenced, or shortly before an instance of the class is created. **Türkçe:** Class'ın ne zaman initialize edileceğini Java Virtual Machine (JVM) belirler; ancak kullanılmadan önce yükleneyeceğini varsayabilirsiniz. Class, program başlarken, static member'larından birine erişildiğinde veya class'ın instance'ı oluşturulmadan hemen önce initialize edilebilir.

English: One of the most important rules with class initialization is that it happens at most once for each class. The class may also never be loaded if it is not used in the program. We summarize the order of initialization for a class as follows: **Türkçe:** Class initialization'ın en önemli kurallarından biri, her class için en fazla bir kez gerçekleşmesidir. Programda hiç kullanılmayan class hiç yüklenmeyebilir. Bir class'ın initialization order'ı şöyledir:

Initialize Class X

Türkçe başlık: Class X'i Initialize Et

English: 1. If there is a superclass Y of X, then initialize class Y first. **Türkçe:** 1. X'in superclass'ı Y varsa önce class Y'yi initialize edin.

English: 2. Process all static variable declarations in the order in which they appear in the class. **Türkçe:** 2. Bütün static variable declaration'larını class'ta gördükleri sırayla işleyin.

English: 3. Process all static initializers in the order in which they appear in the class. **Türkçe:** 3. Bütün static initializer'ları class'ta gördükleri sırayla işleyin.

English: Taking a look at an example, what does the following program print? **Türkçe:** Bir örneğe baktığımızda aşağıdaki program neyi yazdırıyor?

```
public class Animal {
    static { System.out.print("A"); }
}
public class Hippo extends Animal {
    public static void main(String[] grass) {
        System.out.print("C");
        new Hippo();
        new Hippo();
        new Hippo();
    }
    static { System.out.print("B"); }
}
```

Source page 0298

English: It prints ABC exactly once. Since the main() method is inside the Hippo class, the class will be initialized first, starting with the superclass and printing AB. Afterward, the main() method is executed, printing C. Even though the main() method creates three instances, the class is loaded only once. **Türkçe:** Program yalnızca bir kez ABC yazdırır. main() method'u Hippo class'ında bulunduğu için önce class hierarchy superclass'tan başlayarak initialize edilir ve AB yazdırılır. Ardından main() çalışır ve C yazdırır. main() üç instance oluştursa da class yalnızca bir kez yüklenir.

Why the Hippo Program Printed C After AB

Türkçe başlık: Hippo Programı Neden AB'den Sonra C Yazdırıldı?

English: In the previous example, the Hippo class was initialized before the main() method was executed. This happened because our main() method was inside the class being executed, so it had to be loaded on startup. What if you instead called Hippo inside another program? **Türkçe:** Önceki örnekte Hippo, main() çalışmadan önce initialize edildi; çünkü program entry point'i bu class'ın içindeydi ve başlangıçta yüklenmesi gerekiyordu. Peki Hippo başka bir programdan çağrılırsa ne olur?

```
public class HippoFriend {  
    public static void main(String[] grass) {  
        System.out.print("C");  
        new Hippo();  
    }  
}
```

English: Assuming the class isn't referenced anywhere else, this program will likely print CAB, with the Hippo class not being loaded until it is needed inside the main() method. We say likely because the rules for when classes are loaded are determined by the JVM at runtime. **Türkçe:** Class'a başka yerde reference verilmediğini varsayarsak program büyük olasılıkla CAB yazdırır; çünkü Hippo, main() içinde gerekene kadar yüklenmez. "Büyük olasılıkla" diyoruz; çünkü class loading zamanını runtime'da JVM belirler.

English: For the exam, you just need to know that a class must be initialized before it is referenced or used. Also, the class containing the program entry point, aka the main() method, is loaded before the main() method is executed. **Türkçe:** Sınav için bir class'ın kendisine reference verilmeden veya kullanılmadan önce initialize edilmesi gerektiğini bilmeniz yeterlidir. Program entry point'ini, yani main() method'unu içeren class da main() çalışmadan önce yüklenir.

Initializing final Fields

Türkçe başlık: final Field'ları Initialize Etmek

English: Before we delve into order of initialization for instance members, we need to talk about final fields (instance variables) for a minute. When we presented instance and class variables in Chapter 1, we told you they are assigned a default value based on their type if no value is specified. For example, a double is initialized with 0.0, while an object reference is initialized to null. A default value is only applied to a non-final field, though. **Türkçe:** Instance member'ların initialization order'ına geçmeden önce **final** field'ları (instance variable'ları) ele almalıyız. Chapter 1'de instance ve class variable'ların explicit value verilmezse type'larına göre default value aldığını gördük. Örneğin `double, 0.0`; object reference ise `null` ile initialize edilir. Fakat default value yalnızca **final** olmayan field'a uygulanır.

English: As you saw in Chapter 5, final static variables must be explicitly assigned a value exactly once. Fields marked final follow similar rules. They can be assigned values in the line in which they are declared or in an instance initializer. **Türkçe:** Chapter 5'te gördüğünüz üzere **static final** variable'lara exactly once (tam bir kez) explicit value atanmalıdır. **final** field'lar da benzer kurallara uyar: Declaration satırında veya instance initializer içinde değer alabilirler.

```
public class MouseHouse {
    private final int volume;
    private final String name = "The Mouse House"; // Declaration assignment
}
```

Source page 0299

```
    volume = 10; // Instance initializer assignment
}
```

English: Unlike static class members, though, final instance fields can also be set in a constructor. The constructor is part of the initialization process, so it is allowed to assign final instance variables. For the exam, you need to know one important rule: by the time the constructor completes, all final instance variables must be assigned a value exactly once. **Türkçe:** Static class member'lardan farklı olarak **final** instance field'lar constructor içinde de atanabilir. Constructor initialization sürecinin parçası olduğu için bu assignment geçerlidir. Sınav kuralı: Constructor tamamlanıncaya kadar bütün **final** instance variable'lara exactly once değer atanmış olmalıdır.

English: Let's try this out in an example: **Türkçe:** Bunu bir örnekte deneyelim:

```
public class MouseHouse {
    private final int volume;
    private final String name;
    public MouseHouse() {
        this.name = "Empty House"; // Constructor assignment
    }
    {
        volume = 10; // Instance initializer assignment
    }
}
```

English: Unlike local final variables, which are not required to have a value unless they are actually used, final instance variables must be assigned a value. If they are not assigned a value when they are declared or in an instance initializer, then they must be assigned a value in the constructor declaration. Failure to do so will result in a compiler error on the line that declares the constructor. **Türkçe:** Yalnızca kullanıldıklarında initialize edilmeleri gereken local `final` variable'ların aksine, `final` instance variable'lara mutlaka değer atanmalıdır. Declaration'da veya instance initializer'da değer almamışlarsa constructor içinde atanmalıdırlar. Aksi durumda compiler error constructor declaration satırında bildirilir.

```
public class MouseHouse {
    private final int volume;
    private final String type;
    {
        this.volume = 10;
    }
    public MouseHouse(String type) {
        this.type = type;
    }
    public MouseHouse() { // DOES NOT COMPILE
        this.volume = 2; // DOES NOT COMPILE
    }
}
```

English: In this example, the first constructor that takes a String argument compiles. In terms of assigning values, each constructor is reviewed individually, which is why the second constructor does not compile. First, the constructor fails to set a value for the type variable. **Türkçe:** Bu örnekte `String` argument alan ilk constructor derlenir. Value assignment bakımından her constructor ayrı incelendiğinden ikinci constructor derlenmez. Birinci sorun, bu constructor'ın `type` variable'ına değer atamamasıdır.

Source page 0300

English: The compiler detects that a value is never set for type and reports an error on the line where the constructor is declared. Second, the constructor sets a value for the volume variable, even though it was already assigned a value by the instance initializer. **Türkçe:** Compiler `type` için hiç değer atanmadığını saptar ve constructor declaration satırında error verir. İkinci sorun, `volume` variable'ına instance initializer tarafından zaten değer verilmişken constructor'ın yeniden değer atamasıdır.

English: On the exam, be wary of any instance variables marked final. Make sure they are assigned a value in the line where they are declared, in an instance initializer, or in a constructor. They should be assigned a value only once, and failure to assign a value is considered a compiler error in the constructor. **Türkçe:** Sınavda `final` instance variable'ları dikkatle izleyin. Declaration satırında, instance initializer'da veya constructor'da değer aldıklarını doğrulayın. Exactly once atanmalıdırlar; hiç atanmamaları constructor'da compiler error oluşturur.

English: What about final instance variables when a constructor calls another constructor in the same class? In that case, you have to follow the flow carefully, making sure every final instance variable is assigned a value exactly once. We can replace our previous bad constructor with the following one that does compile: **Türkçe:** Constructor aynı class'taki başka bir constructor'ı çağırıyorsa akışı dikkatle izleyin ve her `final` instance variable'ın exactly once değer aldığını doğrulayın. Önceki hatalı constructor yerine aşağıdaki derlenen sürümü kullanabiliriz:

```
public MouseHouse() {  
    this(null);  
}
```

English: This constructor does not perform any assignments to any final instance variables, but it calls the `MouseHouse(String)` constructor, which we observed compiles without issue. **Türkçe:** Bu constructor doğrudan hiçbir `final` instance variable'a assignment yapmaz; ancak sorunsuz derlendiğini gördüğümüz `MouseHouse(String)` constructor'ını çağırır.

English: We use `null` here to demonstrate that the variable does not need to be an object value. We can assign a `null` value to final instance variables as long as they are explicitly set. **Türkçe:** Buradaki `null`, variable'ın gerçek bir object value taşımak zorunda olmadığını gösterir. Explicit biçimde atandığı sürece `final` instance variable'a `null` verilebilir.

Initializing Instances

Türkçe başlık: Instance'ları Initialize Etmek

English: We've covered class initialization and final fields, so now it's time to move on to order of initialization for objects. We'll warn you that this can be a bit cumbersome at first, but the exam isn't likely to ask questions more complicated than the examples in this section. We promise to take it slowly, though. **Türkçe:** Class initialization ile `final` field'ları gördük; şimdi object initialization order'a geçiyoruz. İlk bakışta karmaşık gelebilir, ancak sınavın bu bölümdeki örneklerden daha zor sorular sorması beklenmez. Adım adım ilerleyeceğiz.

English: First, start at the lowest-level constructor where the `new` keyword is used. Remember, the first line of every constructor is a call to `this()` or `super()`, and if omitted, the compiler will automatically insert a call to the parent no-argument constructor `super()`. Then, progress upward and note the order of constructors. Finally, initialize each class starting with the superclass, processing each instance initializer and constructor in the reverse order in which it was called. We summarize the order of initialization for an instance as follows: **Türkçe:** Önce `new` kullanılan en alt düzey constructor'dan başlayın. Her constructor'ın first statement'ı `this()` veya `super()` çağrısıdır; yazılmamışsa compiler no-argument parent constructor'a `super()` çağrısı ekler. Çağrı zincirini yukarı doğru izleyip constructor sırasını belirleyin. Ardından superclass'tan başlayarak her class'ın instance initializer'larını ve constructor'ını, çağrı zincirinin çözülme sırasıyla işleyin. Instance initialization order şöyledir:

Initialize Instance of X

Türkçe başlık: X Instance'ını Initialize Et

English: 1. Initialize class X if it has not been previously initialized. **Türkçe:** 1. Daha önce initialize edilmemişse class X'i initialize edin.

English: 2. If there is a superclass Y of X, then initialize the instance of Y first. **Türkçe:** 2. X'in superclass'ı Y varsa önce Y instance'ını initialize edin.

English: 3. Process all instance variable declarations in the order in which they appear in the class. **Türkçe:** 3. Bütün instance variable declaration'larını class'ta göründükleri sırayla işleyin.

English: 4. Process all instance initializers in the order in which they appear in the class. **Türkçe:** 4. Bütün instance initializer'ları class'ta göründükleri sırayla işleyin.

English: 5. Initialize the constructor, including any overloaded constructors referenced with this(). **Türkçe:** 5. `this()` ile çağrılan overloaded constructor'lar dahil constructor'ı çalıştırın.

Source page 0301

English: Let's try an example with no inheritance. See if you can figure out what the following application outputs: **Türkçe:** Inheritance içermeyen bir örnekle başlayalım. Aşağıdaki uygulamanın çıktısını bulmaya çalışın:

```
1: public class ZooTickets {
2:     private String name = "BestZoo";
3:     { System.out.print(name + "- "); }
4:     private static int COUNT = 0;
5:     static { System.out.print(COUNT + "- "); }
6:     static { COUNT += 10; System.out.print(COUNT + "- "); }
7:
8:     public ZooTickets() {
9:         System.out.print("z- ");
10:    }
11:
12:    public static void main(String... patrons) {
13:        new ZooTickets();
14:    } }
```

English: The output is as follows: 0- 10- BestZoo- z- **Türkçe:** Çıktı şöyledir: 0- 10- BestZoo- z-

English: First, we have to initialize the class. Since there is no superclass declared, which means the superclass is Object, we can start with the static components of ZooTickets. In this case, lines 4, 5, and 6 are executed, printing 0- and 10- . Next, we initialize the instance created on line 13. Again, since no superclass is declared, we start with the instance components. Lines 2 and 3 are executed, which prints BestZoo- . Finally, we run the constructor on lines 8–10, which outputs z- . **Türkçe:** Önce class initialize edilir. Explicit superclass belirtilmediği, dolayısıyla superclass `Object` olduğu için `ZooTickets`ın static component'ları işlenir: Line 4, 5 ve 6 çalışır; 0- ile 10- yazdırılır. Ardından line 13'te oluşturulan instance initialize edilir. Line 2 ve 3 çalışıp `BestZoo-` yazdırır. Son olarak line 8–10'daki constructor çalışır ve `z-` yazdırır.

English: Next, let's try a simple example with inheritance: **Türkçe:** Şimdi inheritance içeren basit bir örnek deneyelim:

```
class Primate {  
    public Primate() {  
        System.out.print("Primate- ");  
    }  
    class Ape extends Primate {  
        public Ape(int fur) {  
            System.out.print("Ape1- ");  
        }  
        public Ape() {  
            System.out.print("Ape2- ");  
        }  
    }  
}
```

Source page 0302

```
public class Chimpanzee extends Ape {  
    public Chimpanzee() {  
        super(2);  
        System.out.print("Chimpanzee- ");  
    }  
    public static void main(String[] args) {  
        new Chimpanzee();  
    }  
}
```

English: The compiler inserts the `super()` command as the first statement of both the `Primate` and `Ape` constructors. The code will execute with the parent constructors called first and yield the following output:

Türkçe: Compiler hem `Primate` hem `Ape` constructor'ının first statement'ı olarak `super()` ekler. Parent constructor'lar önce çalışır ve çıktı şöyle olur:

Primate- Ape1- Chimpanzee-

Türkçe başlık: `Primate- Ape1- Chimpanzee-`

English: Notice that only one of the two `Ape()` constructors is called. You need to start with the call to `new Chimpanzee()` to determine which constructors will be executed. Remember, **Türkçe:** İki `Ape()` constructor'ından yalnızca birinin çağrıldığına dikkat edin. Hangi constructor'ların çalışacağını bulmak için `new Chimpanzee()` çağrısından başlayıp zinciri izlemelisiniz. Unutmayın:

English: constructors are executed from the bottom up, but since the first line of every constructor is a call to another constructor, the flow ends up with the parent constructor executed before the child constructor.

Türkçe: Constructor çağrılarını en alttan başlayarak izleriz; fakat her constructor'ın first statement'ı başka bir constructor çağrısı olduğundan gerçekte parent constructor, child constructor'dan önce çalışır.

English: The next example is a little harder. What do you think happens here? **Türkçe:** Sonraki örnek biraz daha zordur. Sizce ne olur?

```
1: public class Cuttlefish {
2: private String name = "swimmy";
3: { System.out.println(name); }
4: private static int COUNT = 0;
5: static { System.out.println(COUNT); }
6: { COUNT++; System.out.println(COUNT); }
7:
8: public Cuttlefish() {
9: System.out.println("Constructor");
10: }
11:
12: public static void main(String[] args) {
13: System.out.println("Ready");
14: new Cuttlefish();
15: } }
```

English: The output looks like this: **Türkçe:** Çıktı şöyledir:

English: 0 **Türkçe:** 0

English: Ready **Türkçe:** Ready

English: swimmy **Türkçe:** swimmy

English: 1 **Türkçe:** 1

English: Constructor **Türkçe:** Constructor

Source page 0303

English: No superclass is declared, so we can skip any steps that relate to inheritance. We first process the static variables and static initializers— lines 4 and 5, with line 5 printing 0. Now that the static initializers are out of the way, the main() method can run, which prints Ready. Next we create an instance declared on line 14. Lines 2, 3, and 6 are processed, with line 3 printing swimmy and line 6 printing 1. Finally, the constructor is run on lines 8–10, which prints Constructor. **Türkçe:** Explicit superclass bulunmadığından inheritance adımlarını atlarız. Önce static variable ve static initializer'lar, yani line 4 ve 5 işlenir; line 5 0 yazdırır. Static initialization tamamlanınca `main()` çalışır ve `Ready` yazdırır. Sonra line 14'teki instance oluşturulur. Line 2, 3 ve 6 işlenir; sırasıyla `swimmy` ve `1` yazdırılır. Son olarak line 8–10'daki constructor çalışır ve `Constructor` yazdırır.

English: Ready for a more difficult example, the kind you might see on the exam? What does the following output? **Türkçe:** Sınavda görebileceğiniz daha zor bir örneğe hazır mısınız? Aşağıdaki kodun çıktısı nedir?

```
1: class GiraffeFamily {
2: static { System.out.print("A"); }
3: { System.out.print("B"); }
4:
5: public GiraffeFamily(String name) {
6: this(1);
7: System.out.print("C");
8: }
9:
10: public GiraffeFamily() {
11: System.out.print("D");
12: }
13:
14: public GiraffeFamily(int stripes) {
15: System.out.print("E");
16: }
17: }
18: public class Okapi extends GiraffeFamily {
19: static { System.out.print("F"); }
20:
21: public Okapi(int stripes) {
22: super("sugar");
23: System.out.print("G");
24: }
25: { System.out.print("H"); }
26:
27: public static void main(String[] grass) {
28: new Okapi(1);
29: System.out.println();
30: new Okapi(2);
31: }
32: }
```

Source page 0304

English: The program prints the following: **Türkçe:** Program aşağıdakileri yazdırır:

English: AFBECHG **Türkçe:** AFBECHG

English: BECHG **Türkçe:** BECHG

English: Let's walk through it. Start with initializing the Okapi class. Since it has a superclass GiraffeFamily, initialize it first, printing A on line 2. Next, initialize the Okapi class, printing F on line 19.

Türkçe: Adım adım ilerleyelim. Önce *Okapi* class'ını initialize ederiz. Superclass'ı *GiraffeFamily* olduğundan önce o initialize edilir ve line 2'de *A* yazdırılır. Ardından *Okapi* initialize edilir ve line 19'da *F* yazdırılır.

English: After the classes are initialized, execute the `main()` method on line 27. The first line of the `main()` method creates a new `Okapi` object, triggering the instance initialization process. Per the first rule, the superclass instance of `GiraffeFamily` is initialized first. Per our third rule, the instance initializer in the superclass `GiraffeFamily` is called, and `B` is printed on line 3. Per the fourth rule, we initialize the constructors. In this case, this involves calling the constructor on line 5, which in turn calls the overloaded constructor on line 14. The result is that `EC` is printed, as the constructor bodies are unwound in the reverse order that they were called. **Türkçe:** Class'lar initialize edildikten sonra line 27'deki `main()` çalışır. `main()` in ilk satırı yeni bir `Okapi` object'i oluşturarak instance initialization sürecini başlatır. İlk kurala göre önce `GiraffeFamily` superclass instance'ı initialize edilir. Üçüncü kurala göre `GiraffeFamily` içindeki instance initializer çağrılır ve line 3'te `B` yazdırılır. Dördüncü kurala göre constructor'lar çalışır: Line 5'teki constructor, line 14'teki overloaded constructor'ı çağırır. Constructor body'leri çağrı sırasının tersine doğru çözülür ve sonuçta `EC` yazdırılır.

English: The process then continues with the initialization of the `Okapi` instance itself. Per the third and fourth rules, `H` is printed on line 25, and `G` is printed on line 23, respectively. The process is a lot simpler when you don't have to call any overloaded constructors. Line 29 then inserts a line break in the output. Finally, line 30 initializes a new `Okapi` object. The order and initialization are the same as line 28, sans the class initialization, so `BECHG` is printed again. Notice that `D` is never printed, as only two of the three constructors in the superclass `GiraffeFamily` are called. **Türkçe:** Ardından `Okapi` instance'ının kendi initialization'ı sürer. Üçüncü ve dördüncü kurala göre line 25'te `H`, line 23'te `G` yazdırılır. Overloaded constructor çağrısı olmadığında süreç çok daha basittir. Line 29 çıktıya newline ekler. Son olarak line 30 yeni bir `Okapi` oluşturur. Class initialization bu kez yapılmaz; diğer sıra line 28 ile aynıdır ve yeniden `BECHG` yazdırılır. `GiraffeFamily` içindeki üç constructor'dan yalnızca ikisi çağrıldığı için `D` hiçbir zaman yazdırılmaz.

English: This example is tricky for a few reasons. There are multiple overloaded constructors, lots of initializers, and a complex constructor pathway to keep track of. Luckily, questions like this are uncommon on the exam. If you see one, just write down what is going on as you read the code. **Türkçe:** Örnek; birden fazla overloaded constructor, çok sayıda initializer ve izlenmesi gereken karmaşık bir constructor path içerdiği için zordur. Bu tür sorular sınavda seyrek. Karşılaşırsanız kodu okurken her adımın yazdığını değeri sırayla not edin.

English: We conclude this section by listing important rules you should know for the exam: **Türkçe:** Sınav için bilmeniz gereken önemli kuralları sıralayarak bu bölümü sonlandırıyoruz:

English: ■ ■ ■ ■ A class is initialized at most once by the JVM before it is referenced or used. **Türkçe:** ■ ■ ■ ■ Bir class, kendisine reference verilmeden veya kullanılmadan önce JVM tarafından en fazla bir kez initialize edilir.

English: All static final variables must be assigned a value exactly once, either when they are declared or in a static initializer. **Türkçe:** Bütün `static final` variable'lara declaration sırasında veya static initializer'da exactly once değer atanmalıdır.

English: ■ ■ All final fields must be assigned a value exactly once, either when they are declared, in an instance initializer, or in a constructor. **Türkçe:** ■ ■ Bütün `final` field'lara declaration'da, instance initializer'da veya constructor'da exactly once değer atanmalıdır.

English: ■ ■ Non-final static and instance variables defined without a value are assigned a default value based on their type. **Türkçe:** ■ ■ Explicit value verilmeden tanımlanan `final` olmayan static ve instance variable'lara type'larına göre default value atanır.

English: ■ ■ Order of initialization is as follows: variable declarations, then initializers, and finally constructors. **Türkçe:** ■ ■ Initialization order: variable declaration'ları, initializer'lar ve son olarak constructor'lar.

Inheriting Members

Türkçe başlık: Member'ları Inherit Etmek

English: Now that we've created a class, what can we do with it? One of Java's biggest strengths is leveraging its inheritance model to simplify code. For example, let's say you have five classes, each of which extends from the `Animal` class. Furthermore, each class defines an `eat()` method **Türkçe:** Bir class oluşturduktan sonra onunla ne yapabiliriz? Java'nın en güçlü yanlarından biri, inheritance modelini kullanarak kodu sadeleştirmesidir. Her biri `Animal` class'ını extend eden beş class olduğunu ve her class'ın aynı implementation'a sahip bir `eat()` method'u tanımladığını düşünün...

Source page 0305

English: with an identical implementation. In this scenario, it's a lot better to define `eat()` once in the `Animal` class than to have to maintain the same method in five separate classes. **Türkçe:** ...Bu durumda aynı method'u beş ayrı class'ta sürdürmek yerine `eat()` i `Animal` class'ında bir kez tanımlamak çok daha uygundur.

English: Inheriting a class not only grants access to inherited methods in the parent class but also sets the stage for collisions between methods defined in both the parent class and the subclass. In this section, we review the rules for method inheritance and how Java handles such scenarios. **Türkçe:** Bir class'tan inheritance almak parent class'taki inherited method'lara erişim sağlar; aynı zamanda parent ve subclass'ta tanımlanan method'lar arasında collision oluşmasına da zemin hazırlar. Bu kısımda method inheritance kurallarını ve Java'nın bu durumları nasıl çözdüğünü inceleyeceğiz.

English: We refer to the ability of an object to take on many different forms as polymorphism. We cover this more in the next chapter, but for now you just need to know that an object can be used in a variety of ways, in part based on the reference variable used to call the object. **Türkçe:** Bir object'in birçok farklı biçim alabilmesine polymorphism denir. Sonraki bölümde ayrıntılandıracağız; şimdilik object'in, ona erişmek için kullanılan reference variable'ın type'ına da bağlı olarak farklı biçimlerde kullanılabildiğini bilmeniz yeterlidir.

Overriding a Method

Türkçe başlık: Bir Method'u Override Etmek

English: What if a method with the same signature is defined in both the parent and child classes? For example, you may want to define a new version of the method and have it behave differently for that subclass. The solution is to override the method in the child class. In Java, overriding a method occurs when a subclass declares a new implementation for an inherited method with the same signature and compatible return type. **Türkçe:** Aynı signature'a sahip bir method hem parent hem child class'ta tanımlanırsa ne olur? Subclass için method'un farklı davranan yeni bir sürümünü isteyebilirsiniz. Çözüm, method'u child class'ta override etmektir. Java'da method overriding; subclass'ın inherited bir method için aynı signature ve compatible return type ile yeni implementation bildirmesidir.

English: Remember that a method signature is composed of the name of the method and method parameters. It does not include the return type, access modifiers, optional specifiers, or any declared exceptions. **Türkçe:** Method signature yalnızca method adı ile method parameter'larından oluşur. Return type, access modifier, optional specifier ve declared exception'lar signature'a dahil değildir.

English: When you override a method, you may still reference the parent version of the method using the super keyword. In this manner, the keywords this and super allow you to select between the current and parent versions of a method, respectively. We illustrate this with the following example: **Türkçe:** Bir method'u override ettikten sonra da `super` keyword'üyle parent sürümüne erişebilirsiniz. `this` current sürümü, `super` parent sürümü seçmenizi sağlar. Aşağıdaki örneğe bakın:

```
public class Marsupial {
    public double getAverageWeight() {
        return 50;
    }
}
public class Kangaroo extends Marsupial {
    public double getAverageWeight() {
        return super.getAverageWeight()+20;
    }
    public static void main(String[] args) {
        System.out.println(new Marsupial().getAverageWeight()); // 50.0
        System.out.println(new Kangaroo().getAverageWeight()); // 70.0
    }
}
```

Source page 0306

English: In this example, the Kangaroo class overrides the getAverageWeight() method but in the process calls the parent version using the super reference. **Türkçe:** Bu örnekte `Kangaroo`, `getAverageWeight()` method'unu override eder; fakat implementation içinde `super` reference'ıyla parent sürümü de çağırır.

Method Overriding Infinite Calls

Türkçe başlık: Method Overriding'de Sonsuz Çağrı

English: You might be wondering whether the use of super in the previous example was required. **Türkçe:** Önceki örnekte super kullanımının gerekli olup olmadığını merak ediyor olabilirsiniz.

English: For example, what would the following code output if we removed the super keyword? **Türkçe:** Örneğin `super` keyword'ünü kaldırırsak aşağıdaki kodun sonucu ne olur?

```
public double getAverageWeight() {  
    return getAverageWeight()+20; // StackOverflowError  
}
```

English: In this example, the compiler would not call the parent `Marsupial` method; it would call the current `Kangaroo` method. The application will attempt to call itself infinitely and produce a `StackOverflowError` at runtime. **Türkçe:** Compiler parent `Marsupial` method'unu değil, current `Kangaroo` method'unu seçer. Method kendisini sonsuza kadar çağırmaya çalıştığı için runtime'da `StackOverflowError` oluşur.

English: To override a method, you must follow a number of rules. The compiler performs the following checks when you override a method: 1. The method in the child class must have the same signature as the method in the parent class. **Türkçe:** Method overriding sırasında compiler şu kontrolleri yapar: 1. Child class'taki method, parent class'taki method ile aynı signature'a sahip olmalıdır.

English: 2. The method in the child class must be at least as accessible as the method in the parent class. **Türkçe:** 2. Child method en az parent method kadar accessible olmalıdır.

English: 3. The method in the child class may not declare a checked exception that is new or broader than the class of any exception declared in the parent class method. **Türkçe:** 3. Child method, parent method'da bildirilmeyen yeni bir checked exception veya parent'takinden broader bir checked exception bildiremez.

English: 4. If the method returns a value, it must be the same or a subtype of the method in the parent class, known as covariant return types. **Türkçe:** 4. Method bir value döndürüyorsa child method'un return type'ı parent method'un return type'ıyla aynı veya onun subtype'ı olmalıdır; buna covariant return type denir.

English: While these rules may seem confusing or arbitrary at first, they are needed for consistency. **Türkçe:** Bu kurallar ilk başta kafa karıştırıcı veya keyfi görünse de tutarlılık açısından gereklidir.

English: Without these rules in place, it is possible to create contradictions within the Java language. **Türkçe:** Bu kurallar uygulanmadan Java dili içinde çelişkiler yaratmak mümkündür.

English: Rule #1: Method Signatures **Türkçe:** Kural #1: Method İmzaları

English: The first rule of overriding a method is somewhat self-explanatory. If two methods have the same name but different signatures, the methods are overloaded, not overridden. Overloaded methods are considered independent and do not share the same polymorphic properties as overridden methods. **Türkçe:** İlk kural açıktır: İki method aynı adı fakat farklı signature'ları taşıyorsa override değil, overload edilmişlerdir. Overloaded method'lar birbirinden bağımsızdır ve overridden method'ların polymorphic özelliklerini paylaşmaz.

Source page 0307

English: We covered overloading a method in Chapter 5, and it is similar to overriding a method, as both involve defining a method using the same name. **Türkçe:** Chapter 5'te method overloading'i gördük. Overloading ile overriding'in ortak yanı, aynı method adını kullanmalarıdır.

English: Overloading differs from overriding in that overloaded methods use a different parameter list. For the exam, it is important that you understand this distinction and that overridden methods have the same signature and a lot more rules than overloaded methods. **Türkçe:** Fark şudur: Overloaded method'ların parameter list'leri, dolayısıyla signature'ları farklıdır; overridden method'ların signature'ı aynıdır ve overriding için çok daha fazla kural vardır. Sınavda bu ayrım önemlidir.

English: Rule #2: Access Modifiers **Türkçe:** Kural #2: Access Modifier'lar

English: What's the purpose of the second rule about access modifiers? Let's try an illustrative example: **Türkçe:** Access modifier'larla ilgili ikinci kural neden gereklidir? Bir örnekle görelim:

```
public class Camel {
    public int getNumberOfHumps() {
        return 1;
    }
}
public class BactrianCamel extends Camel {
    private int getNumberOfHumps() { // DOES NOT COMPILE
        return 2;
    }
}
```

English: In this example, BactrianCamel attempts to override the getNumberOfHumps() method defined in the parent class but fails because the access modifier private is more restrictive than the one defined in the parent version of the method. Let's say BactrianCamel was allowed to compile, though. Would this class compile? **Türkçe:** BactrianCamel, parent class'taki getNumberOfHumps() method'unu override etmeye çalışır; ancak child sürümdeki private, parent sürümdeki public'dan daha restrictive olduğu için kod derlenmez. Varsayalım ki BactrianCamel'in derlenmesine izin verildi; aşağıdaki class'ın davranışı belirlenebilir miydi?

```
public class Rider {
    public static void main(String[] args) {
        Camel c = new BactrianCamel();
        System.out.print(c.getNumberOfHumps()); // ???
    }
}
```

English: The answer is, we don't know. The reference type for the object is Camel, where the method is declared public, but the object is actually an instance of type BactrianCamel, where the method is declared private. Java avoids these types of ambiguity problems by limiting overriding a method to access modifiers that are as accessible or more accessible than the version in the inherited method. **Türkçe:** Cevap belirsiz olurdu: Object'in reference type'ı Camel ve method burada public; fakat runtime object BactrianCamel instance'ı ve method burada private olurdu. Java bu ambiguity'yi, overriding method'un access level'inin inherited sürümle aynı veya daha geniş olmasını zorunlu tutarak önler.

English: Rule #3: Checked Exceptions **Türkçe:** Kural #3: Checked Exception'lar

English: The third rule says that overriding a method cannot declare new checked exceptions or checked exceptions broader than the inherited method. This is done for polymorphic reasons **Türkçe:** Üçüncü kurala göre overriding method, yeni bir checked exception veya inherited method'dakinden broader bir checked exception bildiremez. Bunun nedeni, access modifier kuralında olduğu gibi polymorphism'dir...

Source page 0308

English: similar to limiting access modifiers. In other words, you could end up with an object that is more restrictive than the reference type it is assigned to, resulting in a checked exception that is not handled or declared. One implication of this rule is that overridden methods are free to declare any number of new unchecked exceptions. **Türkçe:** ...Aksi halde object, atandığı reference type'ın sözleşmesinden daha restrictive olabilir ve handle ya da declare edilmemiş bir checked exception ortaya çıkabilir. Bu sınırlama unchecked exception'lara uygulanmaz; overridden method istediği sayıda yeni unchecked exception bildirebilir.

English: If you don't know what a checked or unchecked exception is, don't worry. **Türkçe:** Checked ve unchecked exception ayrımını henüz bilmiyorsanız endişelenmeyin.

English: We cover this in Chapter 11, "Exceptions and Localization." For now, you just need to know that the rule applies only to checked exceptions. It's also helpful to know that both `IOException` and `FileNotFoundException` are checked exceptions and that `FileNotFoundException` is a subclass of `IOException`. **Türkçe:** Konu Chapter 11, "Exceptions and Localization"da ayrıntılı ele alınır. Şimdilik kuralın yalnızca checked exception'lara uygulandığını bilin. Ayrıca `IOException` ile `FileNotFoundException` checked exception'dır ve `FileNotFoundException`, `IOException`ın subclass'ıdır.

English: Let's try an example: **Türkçe:** Bir örnek deneyelim:

```
public class Reptile {
    protected void sleep() throws IOException {}
    protected void hide() {}
    protected void exitShell() throws FileNotFoundException {}
}

public class GalapagosTortoise extends Reptile {
    public void sleep() throws FileNotFoundException {}
    public void hide() throws FileNotFoundException {} // DOES NOT COMPILE
    public void exitShell() throws IOException {} // DOES NOT COMPILE
}
```

English: In this example, we have three overridden methods. These overridden methods use the more accessible public modifier, which is allowed per our second rule for overridden methods. The first overridden method `sleep()` in `GalapagosTortoise` compiles without issue because the declared exception is narrower than the exception declared in the parent class. **Türkçe:** Örnekte üç overridden method vardır. Child sürümler daha accessible olan `public` modifier'ını kullanır; ikinci overriding kuralına göre bu geçerlidir. `GalapagosTortoise.sleep()` sorunsuz derlenir; çünkü bildirdiği `FileNotFoundException`, parent method'daki `IOException`dan narrower'dır.

English: The overridden `hide()` method does not compile because it declares a new checked exception not present in the parent declaration. The overridden `exitShell()` also does not compile, since `IOException` is a broader checked exception than `FileNotFoundException`. We revisit these exception classes, including memorizing which ones are subclasses of each other, in Chapter 11. **Türkçe:** Overridden `hide()` parent declaration'da bulunmayan yeni bir checked exception bildirdiği için derlenmez. Overridden `exitShell()` de derlenmez; çünkü `IOException`, `FileNotFoundException`dan broader bir checked exception'dır. Exception hierarchy'leri Chapter 11'de yeniden ele alınır.

Source page 0309

English: Rule #4: Covariant Return Types **Türkçe:** Kural #4: Covariant Return Type'lar

English: The fourth and final rule around overriding a method is probably the most complicated, as it requires knowing the relationships between the return types. The overriding method must use a return type that is covariant with the return type of the inherited method. **Türkçe:** Dördüncü ve son overriding kuralı return type'lar arasındaki ilişkiyi bilmeyi gerektirir. Overriding method'un return type'ı, inherited method'un return type'ıyla covariant olmalıdır.

English: Let's try an example for illustrative purposes: **Türkçe:** Açıklama amacıyla bir örnek deneyelim:

```
public class Rhino {
    protected CharSequence getName() {
        return "rhino";
    }
    protected String getColor() {
        return "grey, black, or white";
    }
}
public class JavanRhino extends Rhino {
    public String getName() {
        return "javan rhino";
    }
    public CharSequence getColor() { // DOES NOT COMPILE
        return "grey";
    }
}
```

English: The subclass `JavanRhino` attempts to override two methods from `Rhino`: `getName()` and `getColor()`. Both overridden methods have the same name and signature as the inherited methods. The overridden methods also have a broader access modifier, `public`, than the inherited methods. Remember, a broader access modifier is acceptable in an overridden method. **Türkçe:** `JavanRhino`, `Rhino`dan inherited `getName()` ve `getColor()` method'larını override etmeye çalışır. İki child method da inherited sürümle aynı ad/signature'a ve daha geniş olan `public` access modifier'ına sahiptir. Overriding method'da access'in genişletilmesi geçerlidir.

English: From Chapter 4, "Core APIs," we learned that `String` implements the `CharSequence` interface, making `String` a subtype of `CharSequence`. Therefore, the return type of `getName()` in `JavanRhino` is covariant with the return type of `getName()` in `Rhino`. **Türkçe:** Chapter 4, "Core APIs"den `String`in `CharSequence` interface'ini implement ettiğini, dolayısıyla `CharSequence`in subtype'ı olduğunu biliyoruz. Bu nedenle `JavanRhino.getName()` return type'ı, `Rhino.getName()` return type'ıyla covariant'tır.

English: On the other hand, the overridden `getColor()` method does not compile because `CharSequence` is not a subtype of `String`. To put it another way, all `String` values are `CharSequence` values, but not all `CharSequence` values are `String` values. For instance, a `StringBuilder` is a `CharSequence` but not a `String`. For the exam, you need to know if the return type of the overriding method is the same as or a subtype of the return type of the inherited method. **Türkçe:** Overridden `getColor()` ise derlenmez; çünkü `CharSequence`, `String`in subtype'ı değildir. Her `String` bir `CharSequence`tir, fakat her `CharSequence` bir `String` değildir; örneğin `StringBuilder`, `CharSequence`tir ama `String` değildir. Sınavda overriding method'un return type'ının inherited method'unkiyle aynı veya onun subtype'ı olup olmadığını kontrol edin.

English: A simple test for covariance is the following: given an inherited return type A and an overriding return type B, can you assign an instance of B to a reference variable for A without a cast? If so, then they are covariant. **Türkçe:** Covariance testi: Inherited return type A, overriding return type B ise B instance'ını cast kullanmadan A reference variable'ına atayabiliyor musunuz? Evetse type'lar covariant'tır.

English: This rule applies to primitive types and object types alike. If one of the return types is `void`, then they both must be `void`, as nothing is covariant with `void` except itself. **Türkçe:** Kural primitive ve object type'lara uygulanır. Return type'lardan biri `void` ise diğeri de `void` olmalıdır; `void` ile yalnızca kendisi covariant'tır.

Source page 0310

English: That's everything you need to know about overriding methods for this chapter. In Chapter 9, "Collections and Generics," we revisit overriding methods involving generics. **Türkçe:** Bu chapter için method overriding konusunda gerekenler bunlardır. Generics içeren override örneklerine Chapter 9, "Collections and Generics"te döneceğiz.

English: There's always more to learn! **Türkçe:** Her zaman öğrenecek daha çok şey vardır!

English: Marking Methods with the `@Override` Annotation An annotation is a metadata tag that provides additional information about your code. **Türkçe:** Method'ları `@Override` Annotation'ıyla İşaretlemek — Annotation, kod hakkında ek bilgi sağlayan bir metadata tag'idir.

English: You can use the `@Override` annotation to tell the compiler that you are attempting to override a method. **Türkçe:** Compiler'a method override etmeyi amaçladığınızı bildirmek için `@Override` annotation'ını kullanabilirsiniz.

```
public class Fish {
    public void swim() {};
}
public class Shark extends Fish {
    @Override
    public void swim() {};
}
```

English: When used correctly, the annotation doesn't impact the code. On the other hand, when used incorrectly, this annotation can prevent you from making a mistake. The following does not compile because of the presence of the `@Override` annotation: **Türkçe:** Doğru kullanıldığında annotation runtime davranışını değiştirmez. Yanlış kullanıldığında ise hatayı compile time'da yakalar. Aşağıdaki kod `@Override` bulunduğu için derlenmez:

```
public class Fish {
    public void swim() {};
}
public class Shark extends Fish {
    @Override
    public void swim(int speed) {}; // DOES NOT COMPILE
}
```

English: The compiler sees that you are attempting a method override and looks for an inherited version of `swim()` that takes an `int` value. Since the compiler doesn't find one, it reports an error. While knowing advanced topics (such as how to create annotations) is not required for the exam, knowing how to use them properly is. **Türkçe:** Compiler bir method override etmeye çalıştığınızı anlar ve `int` alan inherited `swim()` sürümünü arar. Böyle bir method bulamayınca error verir. Annotation oluşturmak gibi ileri konular sınav kapsamında değildir; fakat annotation'ları doğru kullanmayı bilmek gerekir.

Source page 0311

English: Redefining private Methods **Türkçe:** private Method'ları Yeniden Bildirmek

English: What happens if you try to override a private method? In Java, you can't override private methods since they are not inherited. Just because a child class doesn't have access to the parent method doesn't mean the child class can't define its own version of the method. It just means, strictly speaking, that the new method is not an overridden version of the parent class's method. **Türkçe:** `private` method override edilemez; çünkü inherit edilmez. Child class parent method'a erişemese de aynı adlı kendi method'unu tanımlayabilir. Ancak bu yeni method, teknik olarak parent method'un overridden sürümü değildir.

English: Java permits you to redeclare a new method in the child class with the same or modified signature as the method in the parent class. This method in the child class is a separate and independent method, unrelated to the parent version's method, so none of the rules for overriding methods is invoked. For example, these two declarations compile: **Türkçe:** Java child class'ta parent'taki `private` method ile aynı ya da farklı signature'a sahip yeni bir method bildirmenize izin verir. Child method parent sürümünden bağımsız olduğundan overriding kuralları uygulanmaz. Örneğin şu iki declaration derlenir:

```
public class Beetle {  
    private String getSize() {  
        return "Undefined";  
    }  
}  
public class RhinocerosBeetle extends Beetle {  
    private int getSize() {  
        return 5;  
    }  
}
```

English: Notice that the return type differs in the child method from String to int. In this example, the method `getSize()` in the parent class is redeclared, so the method in the child class is a new method and not an override of the method in the parent class. **Türkçe:** Child method'un return type'inin `String` yerine `int` olduğuna dikkat edin. Parent `getSize()` method'u `private` olduğundan child declaration yeni bir method'dur; parent method'u override etmez.

English: What if `getSize()` method was declared public in Beetle? In this case, the method in RhinocerosBeetle would be an invalid override. The access modifier in RhinocerosBeetle is more restrictive, and the return types are not covariant. **Türkçe:** `Beetle.getSize()` `public` olsaydı `RhinocerosBeetle` içindeki method geçersiz bir override olurdu: Child access modifier daha restrictive ve return type'lar covariant değildir.

Hiding Static Methods

Türkçe başlık: Static Method'ları Hide Etmek

English: A static method cannot be overridden because class objects do not inherit from each other in the same way as instance objects. On the other hand, they can be hidden. A hidden method occurs when a child class defines a static method with the same name and signature as an inherited static method defined in a parent class. Method hiding is similar to but not exactly the same as method overriding. The previous four rules for overriding a method must be followed when a method is hidden. In addition, a new fifth rule is added for hiding a method: 5. The method defined in the child class must be marked as static if it is marked as static in a parent class. **Türkçe:** Static method override edilemez; çünkü class object'leri instance object'ler gibi birbirinden inheritance almaz. Bunun yerine static method hide edilebilir. Child class, parent'tan inherited static method ile aynı ad ve signature'a sahip static method tanımlarsa method hiding oluşur. Hiding, overriding'e benzer ama aynı değildir: Önceki dört overriding kuralı yine uygulanır ve beşinci kural eklenir: 5. Parent method `static` ise child method da `static` olmalıdır.

English: Put simply, it is method hiding if the two methods are marked static and method overriding if they are not marked static. If one is marked static and the other is not, the class will not compile. **Türkçe:** Kısaca iki method da `static` ise hiding, ikisi de instance method ise overriding söz konusudur. Yalnızca biri `static` ise class derlenmez.

Source page 0312

English: Let's review some examples of the new rule: **Türkçe:** Yeni kuralın bazı örneklerini inceleyelim:

```
public class Bear {
    public static void eat() {
        System.out.println("Bear is eating");
    }
}
public class Panda extends Bear {
    public static void eat() {
        System.out.println("Panda is chewing");
    }
    public static void main(String[] args) {
        eat();
    }
}
```

English: In this example, the code compiles and runs. The eat() method in the Panda class hides the eat() method in the Bear class, printing "Panda is chewing" at runtime. Because they are both marked as static, this is not considered an overridden method. That said, there is still some inheritance going on. If you remove the eat() declaration in the Panda class, then the program prints "Bear is eating" instead. **Türkçe:** Kod derlenir ve çalışır. `Panda.eat()`, `Bear.eat()` method'unu hide eder ve runtime'da `Panda is chewing` yazdırılır. İki method da `static` olduğu için bu overriding değildir. Yine de inheritance vardır: `Panda` içindeki `eat()` declaration kaldırılırsa program inherited sürümünü çağırıp `Bear is eating` yazdırır.

English: See if you can figure out why each of the method declarations in the SunBear class does not compile: **Türkçe:** `SunBear` class'ındaki her method declaration'ın neden derlenmediğini bulmaya çalışın:

```
public class Bear {
    public static void sneeze() {
        System.out.println("Bear is sneezing");
    }
    public void hibernate() {
        System.out.println("Bear is hibernating");
    }
    public static void laugh() {
        System.out.println("Bear is laughing");
    }
}
public class SunBear extends Bear {
    public void sneeze() { // DOES NOT COMPILE
        System.out.println("Sun Bear sneezes quietly");
    }
    public static void hibernate() { // DOES NOT COMPILE
        System.out.println("Sun Bear is going to sleep");
    }
}
```

Source page 0313

```
protected static void laugh() { // DOES NOT COMPILE
    System.out.println("Sun Bear is laughing");
}
}
```

English: In this example, `sneeze()` is marked static in the parent class but not in the child class. The compiler detects that you're trying to override using an instance method. However, `sneeze()` is a static method that should be hidden, causing the compiler to generate an error. The second method, `hibernate()`, does not compile for the opposite reason. The method is marked static in the child class but not in the parent class. **Türkçe:** Bu örnekte `sneeze()` parent class'ta `static`, child class'ta ise instance method'dur. Compiler, static method'u instance method ile override etmeye çalıştığınızı saptar; oysa `sneeze()` hide edilmeliydi ve bu nedenle error verir. İkinci method `hibernate()` tam tersi nedenle derlenmez: Child sürüm `static`, parent sürüm ise instance method'dur.

English: Finally, the `laugh()` method does not compile. Even though both versions of the method are marked static, the version in `SunBear` has a more restrictive access modifier than the one it inherits, and it breaks the second rule for overriding methods. Remember, the four rules for overriding methods must be followed when hiding static methods. **Türkçe:** Son olarak `laugh()` derlenmez. İki sürüm de `static` olsa da `SunBear` sürümünün access modifier'ı inherited sürümden daha restrictive'tir; bu, overriding'in ikinci kuralını ihlal eder. Static method hiding sırasında dört overriding kuralının da geçerli olduğunu unutmayın.

Hiding Variables

Türkçe başlık: Variable'ları Hide Etmek

English: As you saw with method overriding, there are a lot of rules when two methods have the same signature and are defined in both the parent and child classes. Luckily, the rules for variables with the same name in the parent and child classes are much simpler. In fact, Java doesn't allow variables to be overridden. Variables can be hidden, though. **Türkçe:** Aynı signature'lı method'lar parent ve child class'ta tanımlandığında birçok overriding kuralı uygulanır. Aynı adlı variable'ların kuralları daha basittir: Java variable overriding'e izin vermez; variable yalnızca hide edilebilir.

English: A hidden variable occurs when a child class defines a variable with the same name as an inherited variable defined in the parent class. This creates two distinct copies of the variable within an instance of the child class: one instance defined in the parent class and one defined in the child class. **Türkçe:** Child class inherited variable ile aynı adlı yeni bir variable tanımlarsa hidden variable oluşur. Child instance içinde iki ayrı variable kopyası bulunur: biri parent, diğeri child class'ta tanımlıdır.

English: As when hiding a static method, you can't override a variable; you can only hide it. Let's take a look at a hidden variable. What do you think the following application prints? **Türkçe:** Static method'da olduğu gibi variable override edilemez, yalnızca hide edilir. Aşağıdaki uygulamanın çıktısı nedir?

```
class Carnivore {
    protected boolean hasFur = false;
}
public class Meerkat extends Carnivore {
    protected boolean hasFur = true;
    public static void main(String[] args) {
        Meerkat m = new Meerkat();
        Carnivore c = m;
        System.out.println(m.hasFur); // true
        System.out.println(c.hasFur); // false
    }
}
```

Source page 0314

English: Confused about the output? Both of these classes define a `hasFur` variable, but with different values. Even though only one object is created by the `main()` method, both variables exist independently of each other. The output changes depending on the reference variable used. **Türkçe:** İki class da farklı değere sahip bir `hasFur` variable'ı tanımlar. `main()` yalnızca bir object oluştursa da iki variable birbirinden bağımsızdır. Hangi değerin okunduğunu kullanan reference variable belirler.

English: If you didn't understand the last example, don't worry. We cover polymorphism in more detail in the next chapter. For now, you just need to know that overriding a method replaces the parent method on all reference variables (other than `super`), whereas hiding a method or variable replaces the member only if a child reference type is used. **Türkçe:** Polymorphism sonraki bölümde ayrıntılandırılır. Şimdilik şunu bilin: Method overriding, `super` dışındaki tüm reference variable'larda parent method'un yerini alır. Method/variable hiding ise member seçimini reference type'a bağlar; child sürüm yalnızca child reference type kullanıldığında seçilir.

Writing final Methods

Türkçe başlık: `final` Method Yazmak

English: We conclude our discussion of method inheritance with a somewhat self-explanatory rule: final methods cannot be overridden. By marking a method final, you forbid a child class from replacing this method. This rule is in place both when you override a method and when you hide a method. In other words, you cannot hide a static method in a child class if it is marked final in the parent class. **Türkçe:** Method inheritance konusunun son kuralı açıktır: `final` method override edilemez. Bir method'u `final` yapmak child class'ın onu değiştirmesini yasaklar. Kural overriding ve hiding için geçerlidir; parent class'ta `final` olan static method child class'ta hide edilemez.

English: Let's take a look at an example: **Türkçe:** Bir örneğe bakalım:

```
public class Bird {
    public final boolean hasFeathers() {
        return true;
    }
    public final static void flyAway() {}
}
public class Penguin extends Bird {
    public final boolean hasFeathers() { // DOES NOT COMPILE
        return false;
    }
    public final static void flyAway() {} // DOES NOT COMPILE
}
```

English: In this example, the instance method `hasFeathers()` is marked as final in the parent class `Bird`, so the child class `Penguin` cannot override the parent method, resulting in a compiler error. The static method `flyAway()` is also marked final, so it cannot be hidden in the subclass. In this example, whether or not the child method uses the final keyword is irrelevant—the code will not compile either way. **Türkçe:** `hasFeathers()` instance method'u parent `Bird` class'ında `final` olduğu için `Penguin` onu override edemez; compiler error oluşur. Static `flyAway()` da `final` olduğundan subclass'ta hide edilemez. Child method'da ayrıca `final` yazılıp yazılmaması sonucu değiştirmez; kod her iki durumda da derlenmez.

English: This rule applies only to inherited methods. For example, if the two methods were marked private in the parent Bird class, then the Penguin class, as defined, would compile. In that case, the private methods would be redeclared, not overridden or hidden. **Türkçe:** Bu kural yalnızca inherited method'lara uygulanır. İki parent method Bird içinde private olsaydı Penguin derlenirdi; çünkü private method'lar override/hide edilmez, child class'ta bağımsız olarak redeclare edilir.

Source page 0315

Creating Abstract Classes

Türkçe başlık: Abstract Class'lar Oluşturmak

English: When designing a model, we sometimes want to create an entity that cannot be instantiated directly. For example, imagine that we have a Canine class with subclasses Wolf, Fox, and Coyote. We want other developers to be able to create instances of the subclasses, but perhaps we don't want them to be able to create a Canine instance. In other words, we want to force all objects of Canine to have a particular type at runtime. **Türkçe:** Bir model tasarlarlarken bazen doğrudan instantiate edilemeyen bir entity oluşturmak isteriz. Wolf, Fox ve Coyote subclass'larına sahip bir Canine class'ı düşünün. Geliştiriciler subclass instance'ları oluşturabilsin, fakat doğrudan Canine instance'ı oluşturamasın. Böylece runtime'daki her Canine object'inin belirli bir concrete type taşımasını zorunlu kılırız.

Introducing Abstract Classes

Türkçe başlık: Abstract Class'lara Giriş

English: Enter abstract classes. An abstract class is a class declared with the abstract modifier that cannot be instantiated directly and may contain abstract methods. Let's take a look at an example based on the Canine data model: **Türkçe:** Burada abstract class devreye girer. Abstract class, abstract modifier'ıyla bildirilen, doğrudan instantiate edilemeyen ve abstract method içerebilen class'tır. Canine modeline dayanan örneğe bakalım:

```
public abstract class Canine {}  
public class Wolf extends Canine {}  
public class Fox extends Canine {}  
public class Coyote extends Canine {}
```

English: In this example, other developers can create instances of Wolf, Fox, or Coyote, but not Canine. Sure, they can pass a variable reference as a Canine, but the underlying object must be a subclass of Canine at runtime. **Türkçe:** Geliştiriciler Wolf, Fox veya Coyote instance'ı oluşturabilir; Canine instance'ı oluşturamaz. Bir reference variable'ın type'i Canine olabilir, fakat underlying object runtime'da mutlaka Canine'in bir subclass instance'ı olmalıdır.

English: But wait, there's more! An abstract class can contain abstract methods. An abstract method is a method declared with the abstract modifier that does not define a body. Put another way, an abstract method forces subclasses to override the method. **Türkçe:** Abstract class abstract method da içerebilir. Abstract method, abstract modifier'ıyla bildirilen ve body tanımlamayan method'dur; subclass'ları bu method'u override etmeye zorlar.

English: Why would we want this? Polymorphism, of course! By declaring a method abstract, we can guarantee that some version will be available on an instance without having to specify what that version is in the abstract parent class. **Türkçe:** Bunun nedeni polymorphism'dir. Bir method'u abstract bildirerek parent class'ta implementation vermeden, her concrete instance'ta method'un bir sürümünün bulunacağını garanti ederiz.

```
public abstract class Canine {
    public abstract String getSound();
    public void bark() { System.out.println(getSound()); }
}
public class Wolf extends Canine {
    public String getSound() {
        return "Woowooooof!";
    }
}
```

Source page 0316

```
public class Fox extends Canine {
    public String getSound() {
        return "Squeak!";
    }
}
```

```
public class Coyote extends Canine {
    public String getSound() {
        return "Roar!";
    }
}
```

English: We can then create an instance of Fox and assign it to the parent type Canine. The overridden method will be used at runtime. **Türkçe:** Bir Fox instance'ı oluşturup Canine supertype reference'ına atayabiliriz. Runtime'da overridden method çalışır.

```
public static void main(String[] p) {
    Canine w = new Fox();
    w.bark(); // Squeak!
}
```

English: ■■■■■ Easy so far. But there are some rules you need to be aware of: Only instance methods can be marked abstract within a class, not variables, constructors, or static methods. **Türkçe:** ■■■■■ Şimdi temel kurallar: Bir class içinde yalnızca instance method'lar `abstract` olabilir; variable, constructor ve static method `abstract` olamaz.

English: An abstract method can only be declared in an abstract class. **Türkçe:** Abstract method yalnızca abstract class içinde bildirilebilir.

English: A non-abstract class that extends an abstract class must implement all inherited abstract methods. **Türkçe:** Abstract class'ı extend eden non-abstract class, inherited bütün abstract method'ları implement etmelidir.

English: Overriding an abstract method follows the existing rules for overriding methods that you learned about earlier in the chapter. **Türkçe:** Abstract method'u override ederken daha önce gördüğünüz bütün overriding kuralları uygulanır.

English: Let's see if you can spot why each of these class declarations does not compile: **Türkçe:** Bakalım bu class bildirimlerinin her birinin neden derlenmediğini anlayabiliyor musunuz:

```
public class FennecFox extends Canine {
    public int getSound() {
        return 10;
    }
}
public class ArcticFox extends Canine {}
public class Direwolf extends Canine {
    public abstract rest();
    public String getSound() {
        return "Roof!";
    }
}
```

Source page 0317

```
public class Jackal extends Canine {
    public abstract String name;
    public String getSound() {
        return "Laugh";
    }
}
```

English: First off, the FennecFox class does not compile because it is an invalid method override.

Türkçe: Önce **FennecFox**: Geçersiz method override ettiği için derlenmez.

English: In particular, the return types are not covariant. The ArcticFox class does not compile because it does not override the abstract getSound() method. The Direwolf class does not compile because it is not abstract but declares an abstract method rest(). Finally, the Jackal class does not compile because variables cannot be marked abstract. **Türkçe:** Return type'lar covariant değildir. **ArcticFox**, abstract **getSound()** method'unu override etmediği için derlenmez. **Direwolf** non-abstract olduğu halde abstract **rest()** method'u bildirdiği için derlenmez. **Jackal** ise variable'lar **abstract** olamayacağı için derlenmez.

English: An abstract class is most commonly used when you want another class to inherit properties of a particular class, but you want the subclass to fill in some of the implementation details. **Türkçe:** Abstract class çoğunlukla başka class'ların ortak özellikleri inherit etmesini, fakat implementation'ın bazı ayrıntılarını subclass'ların tamamlamasını istediğinizde kullanılır.

English: Earlier, we said that an abstract class is one that cannot be instantiated. This means that if you attempt to instantiate it, the compiler will report an exception, as in this example: **Türkçe:** Abstract class instantiate edilemez. Bunu yapmaya çalışırsanız aşağıdaki gibi runtime exception değil, compiler error alırsınız:

```
abstract class Alligator {
    public static void main(String... food) {
        var a = new Alligator(); // DOES NOT COMPILE
    }
}
```

English: An abstract class can be initialized, but only as part of the instantiation of a non-abstract subclass. **Türkçe:** Abstract class initialize edilebilir; ancak yalnızca non-abstract subclass'ın instantiation sürecinin bir parçası olarak.

Declaring Abstract Methods

Türkçe başlık: Abstract Method Bildirmek

English: An abstract method is always declared without a body. It also includes a semicolon (;) after the method declaration. As you saw in the previous example, an abstract class may include non-abstract methods, in this case with the bark() method. In fact, an abstract class can include all of the same members as a non-abstract class, including variables, static and instance methods, constructors, etc.

Türkçe: Abstract method her zaman body olmadan bildirilir ve declaration sonunda semicolon (;) bulunur. Önceki örnekteki bark() gibi abstract class non-abstract method da içerebilir. Aslında variable, static/instance method ve constructor dahil non-abstract class'taki bütün member türleri abstract class'ta da bulunabilir.

English: It might surprise you to know that an abstract class is not required to include any abstract methods. For example, the following code compiles even though it doesn't define any abstract methods:

Türkçe: Abstract class'ın abstract method içermesi zorunlu değildir. Örneğin aşağıdaki kod hiçbir abstract method tanımlamadığı halde derlenir:

```
public abstract class Llama {
    public void chew() {}
}
```

English: Even without abstract methods, the class cannot be directly instantiated. For the exam, keep an eye out for abstract methods declared outside abstract classes, such as the following: **Türkçe:** Abstract method bulunmasa da class doğrudan instantiate edilemez. Sınavda abstract class dışında bildirilen abstract method'lara dikkat edin:

```
public class Egret { // DOES NOT COMPILE
    public abstract void peck();
}
```


Source page 0318

English: The exam creators like to include invalid class declarations, mixing non-abstract classes with abstract methods. **Türkçe:** Sınav soruları non-abstract class ile abstract method'u karıştıran geçersiz declaration'ları sık kullanır.

English: Like the final modifier, the abstract modifier can be placed before or after the access modifier in class and method declarations, as shown in this Tiger class: **Türkçe:** `final` gibi `abstract` modifier'ı da class ve method declaration'ında access modifier'dan önce veya sonra gelebilir:

```
abstract public class Tiger {  
    abstract public int claw();  
}
```

English: The abstract modifier cannot be placed after the class keyword in a class declaration or after the return type in a method declaration. The following Bear and howl() declarations do not compile for these reasons: **Türkçe:** `abstract`, class declaration'ında `class` keyword'ünden; method declaration'ında return type'tan sonra gelemez. Aşağıdaki `Bear` ve `howl()` bu nedenle derlenmez:

```
public class abstract Bear { // DOES NOT COMPILE  
    public int abstract howl(); // DOES NOT COMPILE  
}
```

English: It is not possible to define an abstract method that has a body or default implementation. You can still define a default method with a body— you just can't mark it as abstract. As long as you do not mark the method as final, the subclass has the option to override the inherited method. **Türkçe:** Abstract method body veya default implementation içeremez. Body içeren regular method tanımlayabilirsiniz; fakat onu `abstract` işaretleyemezsiniz. Method `final` değilse subclass inherited method'u isterse override edebilir.

Creating a Concrete Class

Türkçe başlık: Concrete Class Oluşturmak

English: An abstract class becomes usable when it is extended by a concrete subclass. A concrete class is a non-abstract class. The first concrete subclass that extends an abstract class is required to implement all inherited abstract methods. This includes implementing any inherited abstract methods from inherited interfaces, as you see in the next chapter. **Türkçe:** Abstract class, concrete subclass tarafından extend edildiğinde kullanılabilir hale gelir. Concrete class, non-abstract class'tır. Abstract class'ı extend eden ilk concrete subclass, inherited bütün abstract method'ları implement etmek zorundadır. Buna sonraki bölümde göreceğiniz inherited interface'lerden gelen abstract method'lar da dahildir.

English: When you see a concrete class extending an abstract class on the exam, check to make sure that it implements all of the required abstract methods. Can you see why the following Walrus class does not compile? **Türkçe:** Sınavda abstract bir class'ı genişleten concrete bir class gördüğünüzde, gerekli tüm abstract method'ları uyguladığından emin olun. Aşağıdaki Walrus class'ının neden derlenmediğini görebiliyor musunuz?

```
public abstract class Animal {  
    public abstract String getName();  
}  
public class Walrus extends Animal {} // DOES NOT COMPILE
```

English: In this example, we see that `Animal` is marked as abstract and `Walrus` is not, making `Walrus` a concrete subclass of `Animal`. Since `Walrus` is the first concrete subclass, it must implement all inherited abstract methods—`getName()` in this example. Because it doesn't, the compiler reports an error with the declaration of `Walrus`. **Türkçe:** `Animal` abstract, `Walrus` non-abstract olduğundan `Walrus`, `Animal`'in concrete subclass'ıdır. İlk concrete subclass olarak bütün inherited abstract method'ları—burada `getName()` i—implement etmelidir. Etmediği için compiler `Walrus` declaration'ında error verir.

English: We highlight the first concrete subclass for a reason. An abstract class can extend a non-abstract class and vice versa. Anytime a concrete class is extending an abstract class, it must **Türkçe:** “İlk concrete subclass” vurgusu önemlidir. Abstract class non-abstract class'ı, non-abstract class da abstract class'ı extend edebilir. Concrete class abstract class'ı extend ettiğinde...

Source page 0319

English: implement all of the methods that are inherited as abstract. Let's illustrate this with a set of inherited classes: **Türkçe:** ...abstract olarak inherited bütün method'ları implement etmelidir. Bunu bir inheritance zinciriyle görelim:

```
public abstract class Mammal {  
    abstract void showHorn();  
    abstract void eatLeaf();  
}  
public abstract class Rhino extends Mammal {  
    void showHorn() {} // Inherited from Mammal  
}  
public class BlackRhino extends Rhino {  
    void eatLeaf() {} // Inherited from Mammal  
}
```

English: In this example, the `BlackRhino` class is the first concrete subclass, while the `Mammal` and `Rhino` classes are abstract. The `BlackRhino` class inherits the `eatLeaf()` method as abstract and is therefore required to provide an implementation, which it does. **Türkçe:** `Mammal` ve `Rhino` abstract; `BlackRhino` ilk concrete subclass'tır. `BlackRhino`, abstract `eatLeaf()` method'unu inherit eder ve implementation vermek zorundadır; kod bunu yapar.

English: What about the `showHorn()` method? Since the parent class, `Rhino`, provides an implementation of `showHorn()`, the method is inherited in the `BlackRhino` as a non-abstract method. **Türkçe:** `showHorn()` için `Rhino` implementation sağladığından method, `BlackRhino` tarafından non-abstract method olarak inherit edilir.

English: For this reason, the BlackRhino class is permitted but not required to override the showHorn() method. The three classes in this example are correctly defined and compile. **Türkçe:** Dolayısıyla BlackRhino, showHorn() u override edebilir ama etmek zorunda değildir. Üç class da doğru tanımlanmıştır ve derlenir.

English: What if we changed the Rhino declaration to remove the abstract modifier? **Türkçe:** abstract modifier'ı kaldırmak için Rhino bildirimini değiştirseydik ne olurdu?

```
public class Rhino extends Mammal { // DOES NOT COMPILE
    void showHorn() {}
}
```

English: By changing Rhino to a concrete class, it becomes the first non-abstract class to extend the abstract Mammal class. Therefore, it must provide an implementation of both the showHorn() and eatLeaf() methods. Since it only provides one of these methods, the modified Rhino declaration does not compile. **Türkçe:** Rhino concrete yapınca abstract Mammal'ı extend eden ilk non-abstract class olur. Bu yüzden hem showHorn() hem eatLeaf() için implementation vermelidir. Yalnızca birini verdiği için yeni declaration derlenmez.

English: Let's try one more example. The following concrete class Lion inherits two abstract methods, getName() and roar(): **Türkçe:** Bir örnek daha deneyelim. Aşağıdaki concrete class Lion, getName() ve roar() olmak üzere iki abstract method'u inheritance alır:

```
public abstract class Animal {
    abstract String getName();
}
public abstract class BigCat extends Animal {
    protected abstract void roar();
}
```

Source page 0320

```
public class Lion extends BigCat {
    public String getName() {
        return "Lion";
    }
    public void roar() {
        System.out.println("The Lion lets out a loud ROAR!");
    }
}
```

English: In this sample code, BigCat extends Animal but is marked as abstract; therefore, it is not required to provide an implementation for the getName() method. The class Lion is not marked as abstract, and as the first concrete subclass, it must implement all of the inherited abstract methods not defined in a parent class. All three of these classes compile successfully. **Türkçe:** BigCat, Animal'ı extend eder fakat abstract olduğu için getName() implementation'ı vermek zorunda değildir. Lion non-abstract ve ilk concrete subclass olduğundan parent class'larda implement edilmemiş bütün inherited abstract method'ları implement eder. Üç class da başarıyla derlenir.

Creating Constructors in Abstract Classes

Türkçe başlık: Abstract Class'larda Constructor Oluşturmak

English: Even though abstract classes cannot be instantiated, they are still initialized through constructors by their subclasses. For example, consider the following program: **Türkçe:** Abstract class doğrudan instantiate edilemese de subclass instantiation sırasında constructor'ı üzerinden initialize edilir. Aşağıdaki programı inceleyin:

```
abstract class Mammal {
    abstract CharSequence chew();
    public Mammal() {
        System.out.println(chew()); // Does this line compile?
    }
}

public class Platypus extends Mammal {
    String chew() { return "yummy!"; }
    public static void main(String[] args) {
        new Platypus();
    }
}
```

English: Using the constructor rules you learned about earlier in this chapter, the compiler inserts a default no-argument constructor into the Platypus class, which first calls super() in the Mammal class. The Mammal constructor is only called when the abstract class is being initialized through a subclass; therefore, there is an implementation of chew() at the time the constructor is called. This code compiles and prints yummy! at runtime. **Türkçe:** Compiler Platypus class'ına default no-argument constructor ekler; bu constructor önce Mammal içindeki super() zincirini çalıştırır. Mammal constructor'ı abstract class bir subclass üzerinden initialize edilirken çağrıldığından o anda chew() için concrete implementation vardır. Kod derlenir ve runtime'da yummy! yazdırır.

English: For the exam, remember that abstract classes are initialized with constructors in the same way as non-abstract classes. For example, if an abstract class does not provide a constructor, the compiler will automatically insert a default no-argument constructor. **Türkçe:** Sınavda abstract class'ların da non-abstract class'larla aynı constructor initialization kurallarına uyduğunu unutmayın. Abstract class constructor bildirmezse compiler ona default no-argument constructor ekler.

Source page 0321

English: The primary difference between a constructor in an abstract class and a non-abstract class is that a constructor in an abstract class can be called only when it is being initialized by a non-abstract subclass. This makes sense, as abstract classes cannot be instantiated. **Türkçe:** Abstract ve non-abstract class constructor'ları arasındaki temel fark şudur: Abstract class constructor'ı yalnızca non-abstract subclass üzerinden initialization sırasında çağrılabilir; çünkü abstract class doğrudan instantiate edilemez.

Spotting Invalid Declarations

Türkçe başlık: Geçersiz Declaration'ları Saptamak

English: We conclude our discussion of abstract classes with a review of potential issues you're more likely to encounter on the exam than in real life. The exam writers are fond of questions with methods marked as abstract for which an implementation is also defined. For example, can you see why each of the following methods does not compile? **Türkçe:** Abstract class konusunu, gerçek koddan çok sınavda karşılaşılabileceğiniz geçersiz declaration'larla bitirelim. Sorularda `abstract` işaretli olduğu halde implementation da verilen method'lar sık kullanılır. Aşağıdaki method'ların neden derlenmediğini bulun:

```
public abstract class Turtle {  
    public abstract long eat() // DOES NOT COMPILE  
    public abstract void swim() {}; // DOES NOT COMPILE  
    public abstract int getAge() { // DOES NOT COMPILE  
        return 10;  
    }  
    public abstract void sleep; public void goInShell(); // DOES NOT COMPILE  
    // DOES NOT COMPILE  
}
```

English: The first method, `eat()`, does not compile because it is marked abstract but does not end with a semicolon (;). The next two methods, `swim()` and `getAge()`, do not compile because they are marked abstract, but they provide an implementation block enclosed in braces ({}). For the exam, remember that an abstract method declaration must end in a semicolon without any braces. The next method, `sleep`, does not compile because it is missing parentheses, (), for method arguments. The last method, `goInShell()`, does not compile because it is not marked abstract and therefore must provide a body enclosed in braces. **Türkçe:** `eat()` abstract olduğu halde semicolon (;) ile bitmediği için derlenmez. `swim()` ve `getAge()` abstract oldukları halde braces ({}) içinde implementation body verdiği için derlenmez. Abstract method declaration body içermez ve semicolon ile biter. `sleep`, method parameter list'i için parentheses (()) bulunmadığından derlenmez. `goInShell()` ise abstract olmadığı halde body vermediği için derlenmez.

English: Make sure you understand why each of the previous methods does not compile and that you can spot errors like these on the exam. If you come across a question on the exam in which a class or method is marked abstract, make sure the class is properly implemented before attempting to solve the problem. **Türkçe:** Bu hataları sınavda hızla saptayabilmelisiniz. Class veya method `abstract` ise sorunun geri kalanını çözmeden önce declaration'ın geçerli olduğunu doğrulayın.

English: abstract and final Modifiers What would happen if you marked a class or method both abstract and final? If you mark something abstract, you intend for someone else to extend or implement it. But if you mark something final, you are preventing anyone from extending or implementing it. These concepts are in direct conflict with each other. **Türkçe:** `abstract` ve `final` Modifier'ları — Bir class veya method hem `abstract` hem `final` olamaz. `abstract`, başkasının extend/implement etmesini gerektirirken `final` bunu yasaklar; kavramlar doğrudan çelişir.

English: Due to this incompatibility, Java does not permit a class or method to be marked both abstract and final. For example, the following code snippet will not compile: **Türkçe:** Bu uyumsuzluk nedeniyle Java aynı class veya method'da `abstract final` birleşimine izin vermez. Aşağıdaki snippet derlenmez:

```
public abstract final class Tortoise { // DOES NOT COMPILE  
    public abstract final void walk(); // DOES NOT COMPILE  
}
```

Source page 0322

English: In this example, neither the class nor the method declarations will compile because they are marked both abstract and final. The exam doesn't tend to use final modifiers on classes or methods often, so if you see them, make sure they aren't used with the abstract modifier. **Türkçe:** Class ve method declaration'ı birlikte `abstract` ve `final` olduğu için ikisi de derlenmez. Sınavda `final` modifier gördüğünüzde yanında `abstract` bulunmadığını kontrol edin.

English: abstract and private Modifiers A method cannot be marked as both abstract and private. This rule makes sense if you think about it. How would you define a subclass that implements a required method if the method is not inherited by the subclass? The answer is that you can't, which is why the compiler will complain if you try to do the following: **Türkçe:** `abstract` ve `private` Modifier'ları — Method hem `abstract` hem `private` olamaz. `private` method inherit edilmediği için subclass zorunlu abstract method'u göremez ve implement edemez. Bu nedenle aşağıdaki kod compiler error verir:

```
public abstract class Whale {
    private abstract void sing(); // DOES NOT COMPILE
}
public class HumpbackWhale extends Whale {
    private void sing() {
        System.out.println("Humpback whale is singing");
    }
}
```

English: In this example, the abstract method `sing()` defined in the parent class `Whale` is not visible to the subclass `HumpbackWhale`. Even though `HumpbackWhale` does provide an implementation, it is not considered an override of the abstract method since the abstract method is not inherited. The compiler recognizes this in the parent class and reports an error as soon as `private` and `abstract` are applied to the same method. **Türkçe:** Parent `Whale` class'ındaki abstract `sing()`, `private` olduğu için `HumpbackWhale` tarafından görülemez. Child implementation verse de inherited bir method olmadığından overriding oluşmaz. Compiler parent declaration'da `private abstract` birleşimini görür görmez error verir.

English: While it is not possible to declare a method abstract and private, it is possible (albeit redundant) to declare a method final and private. **Türkçe:** Method `abstract private` olamaz; fakat redundant (gereksiz) olsa da `final private` olabilir.

English: If we changed the access modifier from `private` to `protected` in the parent class `Whale`, would the code compile? **Türkçe:** Parent `Whale` class'ındaki access modifier'ı `private` dan `protected` a değiştirirsek kod derlenir mi?

```
public abstract class Whale {
    protected abstract void sing();
}
public class HumpbackWhale extends Whale {
    private void sing() { // DOES NOT COMPILE
        System.out.println("Humpback whale is singing");
    }
}
```

English: In this modified example, the code will still not compile, but for a completely different reason. If you remember the rules for overriding a method, the subclass cannot reduce the **Türkçe:** Kod yine derlenmez, fakat bu kez farklı nedenle: Overriding kurallarına göre subclass parent method'un...

Source page 0323

English: visibility of the parent method, `sing()`. Because the method is declared protected in the parent class, it must be marked as protected or public in the child class. Even with abstract methods, the rules for overriding methods must be followed. **Türkçe:** ...visibility'sini daraltamaz. `sing()` parent class'ta `protected` olduğundan child sürüm de `protected` veya `public` olmalıdır. Abstract method overriding'de de normal overriding kuralları geçerlidir.

English: abstract and static Modifiers As we discussed earlier in the chapter, a static method can only be hidden, not overridden. It is defined as belonging to the class, not an instance of the class. If a static method cannot be overridden, then it follows that it also cannot be marked abstract since it can never be implemented. For example, the following class does not compile: **Türkçe:** `abstract` ve `static` Modifier'ları — Static method class'a aittir, instance'a değil; bu nedenle yalnızca hide edilebilir, override edilemez. Override edilemeyen method implement edilemeyeceği için `abstract` da olamaz. Aşağıdaki class derlenmez:

```
abstract class Hippopotamus {
    abstract static void swim(); // DOES NOT COMPILE
}
```

English: For the exam, make sure you know which modifiers can and cannot be used with one another, especially for abstract classes and interfaces. **Türkçe:** Sınav için, özellikle abstract class'lar ve interface'ler için hangi modifier'ların birbiriyle kullanılıp kullanılamayacağını bildiğinizden emin olun.

Creating Immutable Objects

Türkçe başlık: Immutable Object'ler Oluşturmak

English: As you might remember from Chapter 4, an immutable object is one that cannot change state after it is created. The immutable objects pattern is an object-oriented design pattern in which an object cannot be modified after it is created. **Türkçe:** Chapter 4'ten hatırlayacağınız üzere immutable object, oluşturulduktan sonra state'i değişmeyen object'tir. Immutable object pattern, object'in creation sonrasında değiştirilemediği bir object-oriented design pattern'dir.

English: Immutable objects are helpful when writing secure code because you don't have to worry about the values changing. They also simplify code when dealing with concurrency since immutable objects can be easily shared between multiple threads. **Türkçe:** Immutable object'ler secure code yazmayı kolaylaştırır; value'ların sonradan değişmesinden kaygılanmazsınız. Birden fazla thread arasında güvenle paylaşılabilirler için concurrency kodunu da sadeleştirirler.

Declaring an Immutable Class

Türkçe başlık: Immutable Class Bildirmek

English: Although there are a variety of techniques for writing an immutable class, you should be familiar with a common strategy for making a class immutable: **Türkçe:** Immutable class yazmanın farklı yolları vardır. Bir class'ı immutable yapmak için kullanılan şu yaygın stratejiyi bilmelisiniz:

English: 1. Mark the class as final or make all of the constructors private. **Türkçe:** 1. Class'ı `final` yapın veya bütün constructor'ları `private` yapın.

English: 2. Mark all the instance variables private and final. **Türkçe:** 2. Bütün instance variable'ları `private final` yapın.

English: 3. Don't define any setter methods. **Türkçe:** 3. Setter method tanımlamayın.

English: 4. Don't allow referenced mutable objects to be modified. **Türkçe:** 4. Reference verilen mutable object'lerin dışarıdan değiştirilmesine izin vermeyin.

English: 5. Use a constructor to set all properties of the object, making a copy if needed. **Türkçe:** 5. Object'in bütün property'lerini constructor'da ayarlayın; gerekirse copy oluşturun.

English: The first rule prevents anyone from creating a mutable subclass. The second and third rules ensure that callers don't make changes to instance variables and are the hallmarks of good encapsulation, a topic we discuss along with records in Chapter 7. **Türkçe:** İlk kural mutable subclass oluşturulmasını önler. İkinci ve üçüncü kural caller'ın instance variable'ları değiştirmesini engeller ve iyi encapsulation'ın temelidir. Encapsulation ile record'lar Chapter 7'de ele alınır.

Source page 0324

English: The fourth rule for creating immutable objects is subtle. Basically, it means you shouldn't expose an accessor (or getter) method for mutable instance fields. Can you see why the following creates a mutable object? **Türkçe:** Dördüncü kural daha inceliklidir: Mutable instance field için iç reference'ı doğrudan döndüren accessor/getter yayınlamamalısınız. Aşağıdaki class'ın neden mutable object ürettiğini bulun:

```
import java.util.*;
public final class Animal { // Not an immutable object declaration
    private final ArrayList<String> favoriteFoods;
    public Animal() {
        this.favoriteFoods = new ArrayList<String>();
        this.favoriteFoods.add("Apples");
    }
    public List<String> getFavoriteFoods() {
        return favoriteFoods;
    } }
```


English: We carefully followed the first three rules, but unfortunately, a malicious caller could still modify our data: **Türkçe:** İlk üç kurala uyulsa da malicious caller veriyi hâlâ değiştirebilir:

```
var zebra = new Animal();
System.out.println(zebra.getFavoriteFoods()); // [Apples]
zebra.getFavoriteFoods().clear();
zebra.getFavoriteFoods().add("Chocolate Chip Cookies");
System.out.println(zebra.getFavoriteFoods()); // [Chocolate Chip Cookies]
```

English: Oh no! Zebras should not eat Chocolate Chip Cookies! It's not an immutable object if we can change its contents! If we don't have a getter for the favoriteFoods object, how do callers access it? Simple: by using delegate or wrapper methods to read the data. **Türkçe:** Zebra chocolate chip cookie yememeli! İçerik değiştirilebiliyorsa object immutable değildir. `favoriteFoods` için doğrudan getter vermeden caller veriyi nasıl okur? Delegate veya wrapper method'larla.

```
import java.util.*;
public final class Animal { // An immutable object declaration
    private final List<String> favoriteFoods;
    public Animal() {
        this.favoriteFoods = new ArrayList<String>();
        this.favoriteFoods.add("Apples");
    }
    public int getFavoriteFoodsCount() {
        return favoriteFoods.size();
    }
}
```

Source page 0325

```
public String getFavoriteFoodsItem(int index) {
    return favoriteFoods.get(index);
} }
```

English: In this improved version, the data is still available. However, it is a true immutable object because the mutable variable cannot be modified by the caller. **Türkçe:** Bu sürümde veri okunabilir; fakat mutable variable caller tarafından değiştirilemediği için object gerçekten immutable'dır.

Copy on Read Accessor Methods

Türkçe başlık: Copy-on-Read Accessor Method'ları

English: Besides delegating access to any private mutable objects, another approach is to make a copy of the mutable object any time it is requested. **Türkçe:** Private mutable object'e erişimi delegate etmek yerine, her istek geldiğinde mutable object'in copy'sini döndürebilirsiniz.

```
public ArrayList<String> getFavoriteFoods() {
    return new ArrayList<String>(this.favoriteFoods);
}
```

English: Of course, changes in the copy won't be reflected in the original, but at least the original is protected from external changes. This can be an expensive operation if called frequently by the caller.

Türkçe: Copy üzerindeki değişiklikler original object'e yansımaz; böylece original external modification'dan korunur. Ancak accessor sık çağrılırsa copy işlemi maliyetli olabilir.

Performing a Defensive Copy

Türkçe başlık: Defensive Copy Oluşturmak

English: So, what's this about the fifth and final rule for creating immutable objects? In designing our class, let's say we want a rule that the data for favoriteFoods is provided by the caller and that it always contains at least one element. This rule is often called an invariant; it is true any time we have an instance of the object. **Türkçe:** Beşinci kuralı ele alalım. `favoriteFoods` verisinin caller tarafından sağlanmasını ve her zaman en az bir element içermesini istediğimizi varsayın. Object'in her instance'ında doğru kalması gereken bu kurala invariant denir.

```
import java.util.*;

public final class Animal { // Not an immutable object declaration
    private final ArrayList<String> favoriteFoods;

    public Animal(ArrayList<String> favoriteFoods) {
        if (favoriteFoods == null || favoriteFoods.size() == 0)
            throw new RuntimeException("favoriteFoods is required");
        this.favoriteFoods = favoriteFoods;
    }

    public int getFavoriteFoodsCount() {
        return favoriteFoods.size();
    }
}
```

Source page 0326

```
public String getFavoriteFoodsItem(int index) {
    return favoriteFoods.get(index);
} }
```

English: To ensure that favoriteFoods is provided, we validate it in the constructor and throw an exception if it is not provided. So is this immutable? Not quite! A malicious caller might be tricky and keep their own secret reference to our favoriteFoods object, which they can modify directly. **Türkçe:** `favoriteFoods` constructor'da validate edilir; verilmemişse exception atılır. Yine de object immutable değildir. Malicious caller, aynı `favoriteFoods` object'ine kendi reference'ını saklayıp onu doğrudan değiştirebilir.

```
var favorites = new ArrayList<String>();
favorites.add("Apples");
var zebra = new Animal(favorites); // Caller still has access to favorites
System.out.println(zebra.getFavoriteFoodsItem(0)); // [Apples]
favorites.clear();
favorites.add("Chocolate Chip Cookies");
System.out.println(zebra.getFavoriteFoodsItem(0)); // [Chocolate Chip Cookies]
```

English: Whoops! It seems like `Animal` is not immutable anymore, since its contents can change after it is created. The solution is to make a copy of the list object containing the same elements. **Türkçe:** `Animal` artık immutable değildir; çünkü içeriği creation sonrasında değişebiliyor. Çözüm, aynı element'leri içeren list object'inin copy'sini constructor'da oluşturmaktır.

```
public Animal(List<String> favoriteFoods) {
    if (favoriteFoods == null || favoriteFoods.size() == 0)
        throw new RuntimeException("favoriteFoods is required");
    this.favoriteFoods = new ArrayList<String>(favoriteFoods);
}
```

English: The copy operation is called a defensive copy because the copy is being made in case other code does something unexpected. It's the same idea as defensive driving: prevent a problem before it exists. With this approach, our `Animal` class is once again immutable. **Türkçe:** Bu işleme defensive copy denir; çünkü başka kodun beklenmedik bir değişiklik yapmasına karşı önlem olarak copy alınır. Defensive driving gibi sorun oluşmadan engellenir. Bu sürümde `Animal` yeniden immutable'dır.

Summary

Türkçe başlık: Özet

English: This chapter took the basic class structures we've presented throughout the book and expanded them by introducing the notion of inheritance. Java classes follow a single-inheritance pattern in which every class has exactly one direct parent class, with all classes eventually inheriting from `java.lang.Object`. **Türkçe:** Bu chapter, temel class yapılarını inheritance kavramıyla genişletti. Java class'ları single inheritance modeline uyar: Her class'ın tam bir direct parent class'ı vardır ve bütün class'lar sonunda `java.lang.Object`ten inheritance alır.

English: Inheriting a class gives you access to all of the public and protected members of the class. It also gives you access to package members of the class if the classes are in the same package. All instance methods, constructors, and instance initializers have access to two special reference variables: `this` and `super`. Both `this` and `super` provide access to **Türkçe:** Bir class'tan inheritance almak onun bütün `public` ve `protected` member'larına; class'lar aynı package'taysa package-access member'larına erişim sağlar. Bütün instance method, constructor ve instance initializer'lar iki özel reference variable'a erişebilir: `this` ve `super`. Her ikisi de...

Source page 0327

English: all inherited members, with only `this` providing access to all members in the current class declaration. **Türkçe:** ...bütün inherited member'lara erişir; current class declaration'ındaki bütün member'lara ise yalnızca `this` erişir.

English: Constructors are special methods that use the class name and do not have a return type. **Türkçe:** Constructor, class adını taşıyan ve return type'ı bulunmayan özel method'dur.

English: They are used to instantiate new objects. Declaring constructors requires following a number of important rules. If no constructor is provided, the compiler will automatically insert a default no-argument constructor in the class. The first line of every constructor is a call to an overloaded constructor, `this()`, or a parent constructor, `super()`; otherwise, the compiler will insert a call to `super()` as the first line of the constructor. In some cases, such as if the parent class does not define a no-argument constructor, this can lead to compilation errors. **Türkçe:** Yeni object instantiate etmek için kullanılır. Hiç constructor yazılmazsa compiler default no-argument constructor ekler. Her constructor'ın first statement'ı `this()` ile overloaded constructor'a veya `super()` ile parent constructor'a çağrıdır; explicit çağrı yoksa compiler `super()` ekler. Parent class'ta no-argument constructor bulunmaması gibi durumlarda bu ekleme compilation error'a yol açabilir.

English: Pay close attention on the exam to any class that defines a constructor with arguments and doesn't define a no-argument constructor. **Türkçe:** Sınavda argument alan constructor tanımlayıp no-argument constructor tanımlamayan class'lara özellikle dikkat edin.

English: Classes are initialized in a predetermined order: superclass initialization; static variables and static initializers in the order that they appear; instance variables and instance initializers in the order they appear; and finally, the constructor. All final instance variables must be assigned a value exactly once. **Türkçe:** Initialization order önceden bellidir: Superclass initialization; source order'da static variable ve static initializer'lar; source order'da instance variable ve instance initializer'lar; son olarak constructor. Bütün `final` instance variable'lara exactly once değer atanmalıdır.

English: We reviewed overloaded, overridden, hidden, and redeclared methods and showed how they differ. A method is overloaded if it has the same name but a different signature as another accessible method. A method is overridden if it has the same signature as an inherited method, with access modifiers, exceptions, and a return type that are compatible. A static method is hidden if it has the same signature as an inherited static method. Finally, a method is redeclared if it has the same name and possibly the same signature as an uninherited method. **Türkçe:** Overloaded, overridden, hidden ve redeclared method'ların farklarını gördük. Accessible başka bir method ile aynı adı fakat farklı signature'ı taşıyan method overloaded'dır. Inherited method ile aynı signature'a ve compatible access modifier, exception ile return type'a sahip method overridden'dır. Inherited static method ile aynı signature'ı taşıyan static method hidden'dır. Inherit edilmeyen method ile aynı adı, hatta aynı signature'ı taşıyabilen yeni method ise redeclared'dır.

English: We then moved on to abstract classes, which are just like regular classes except that they cannot be instantiated and may contain abstract methods. An abstract class can extend a non-abstract class and vice versa. Abstract classes can be used to define a framework that other developers write subclasses against. An abstract method is one that does not include a body when it is declared. An abstract method can only be placed inside an abstract class or interface. Next, an abstract method can be overridden with another abstract declaration or a concrete implementation, provided the rules for overriding methods are followed. The first concrete class must implement all of the inherited abstract methods, whether they are inherited from an abstract class or an interface. **Türkçe:** Abstract class regular class'a benzer; ancak instantiate edilemez ve abstract method içerebilir. Abstract ile non-abstract class birbirini extend edebilir. Abstract class, geliştiricilerin subclass yazacağı bir framework tanımlar. Abstract method body içermez ve yalnızca abstract class veya interface içinde bulunur. Overriding kuralları korunarak başka bir abstract declaration ya da concrete implementation ile override edilebilir. İlk concrete class, abstract class veya interface'ten gelen bütün inherited abstract method'ları implement etmelidir.

English: Finally, this chapter showed you how to create immutable objects in Java. Although there are a number of different techniques to do so, we included the most common one you should know for the exam. Immutable objects are extremely useful in practice, especially in multi-threaded applications, since they do not change. **Türkçe:** Son olarak Java'da immutable object oluşturmanın sınav için bilmeniz gereken yaygın tekniğini gördük. State'leri değişmediği için immutable object'ler özellikle multithreaded application'larda çok kullanışlıdır.

Exam Essentials

Türkçe başlık: Sınav Esasları

English: Be able to write code that extends other classes. A Java class that extends another class inherits all of its public and protected methods and variables. If the class is in the same package, it also inherits all package members of the class. Classes that are marked final cannot be extended. Finally, all classes in Java extend `java.lang.Object` either directly or from a superclass. **Türkçe:** Başka class'ları extend eden kod yazabilin. Bir Java class'ı extend ettiği class'ın bütün `public` ve `protected` method/variable'larını; aynı package içindeyse package member'larını inherit eder. `final` class extend edilemez. Java'daki bütün class'lar doğrudan ya da superclass zinciri üzerinden `java.lang.Object` i extend eder.

Source page 0328

English: Be able to distinguish and use `this`, `this()`, `super`, and `super()`. To access a current or inherited member of a class, the `this` reference can be used. To access an inherited member, the `super` reference can be used. The `super` reference is often used to reduce ambiguity, such as when a class reuses the name of an inherited method or variable. The calls to `this()` and `super()` are used to access constructors in the same class and parent class, respectively. **Türkçe:** `this`, `this()`, `super` ve `super()` ayrımını yapıp kullanabilin. `this`, current veya inherited member'a; `super` yalnızca inherited member'a erişir. `super`, inherited method/variable adının child class'ta yeniden kullanılması gibi ambiguity durumlarını çözer. `this()` aynı class'taki, `super()` parent class'taki constructor'ı çağırır.

English: Evaluate code involving constructors. The first line of every constructor is a call to another constructor within the class using `this()` or a call to a constructor of the parent class using the `super()` call. The compiler will insert a call to `super()` if no constructor call is declared. If the parent class doesn't contain a no-argument constructor, an explicit call to the parent constructor must be provided. Be able to recognize when the default constructor is provided. Remember that the order of initialization is to initialize all classes in the class hierarchy, starting with the superclass. Then the instances are initialized, again starting with the superclass. All final variables must be assigned a value exactly once by the time the constructor is finished. **Türkçe:** Constructor içeren kodu değerlendirebilin. Her constructor'ın first statement'ı `this()` ile same-class constructor veya `super()` ile parent constructor çağırısıdır. Explicit constructor call yoksa compiler `super()` ekler. Parent class'ta no-argument constructor yoksa mevcut parent constructor'a explicit call gerekir. Default constructor'ın ne zaman eklendiğini bilin. Önce class hierarchy superclass'tan başlayarak, sonra instance'lar yine superclass'tan başlayarak initialize edilir. Constructor bitene kadar bütün `final` variable'lar exactly once atanmalıdır.

English: Understand the rules for method overriding. Java allows methods to be overridden, or replaced, by a subclass if certain rules are followed: a method must have the same signature, be at least as accessible as the parent method, must not declare any new or broader exceptions, and must use covariant return types. Methods marked final may not be overridden or hidden. **Türkçe:** Method overriding kurallarını anlayın. Child method aynı signature'a sahip olmalı, parent method kadar accessible olmalı, yeni/broader checked exception bildirmemeli ve covariant return type kullanılmalıdır. `final` method override veya hide edilemez.

English: Recognize the difference between method overriding and method overloading. Both method overloading and overriding involve creating a new method with the same name as an existing method. When the method signature is the same, it is referred to as method overriding and must follow a specific set of override rules to compile. When the method signature is different, with the method taking different inputs, it is referred to as method overloading, and none of the override rules are required. Method overriding is important to polymorphism because it replaces all calls to the method, even those made in a superclass. **Türkçe:** Method overriding ile overloading'i ayırın. İkisinde de mevcut method ile aynı ad kullanılır. Signature aynıysa overriding'dir ve overriding kuralları uygulanır; input'lar dolayısıyla signature farklıysa overloading'dir ve overriding kuralları uygulanmaz. Overriding polymorphism için önemlidir; superclass üzerinden yapılanlar dahil method çağrılarını runtime object sürümüne yönlendirir.

English: Understand the rules for hiding methods and variables. When a static method is overridden in a subclass, it is referred to as method hiding. Likewise, variable hiding is when an inherited variable name is reused in a subclass. In both situations, the original method or variable still exists and is accessible depending on where it is accessed and the reference type used. For method hiding, the use of static in the method declaration must be the same between the parent and child class. Finally, variable and method hiding should generally be avoided since it leads to confusing and difficult-to-follow code. **Türkçe:** Method/variable hiding kurallarını anlayın. Child class'ta inherited static method ile aynı signature'a sahip static method tanımlamak method hiding'dir. Inherited variable adını child class'ta yeniden kullanmak variable hiding'dir. Original member varlığını korur; seçilen member erişim yeri ve reference type'a bağlıdır. Method hiding'de parent ve child declaration'ın ikisi de `static` olmalıdır. Hiding kafa karıştırıcı kod ürettiği için genellikle kaçınılmalıdır.

English: Be able to write code that creates and extends abstract classes. In Java, classes and methods can be declared as abstract. An abstract class cannot be instantiated. An instance of an abstract class can be obtained only through a concrete subclass. Abstract classes can include any number of abstract and non-abstract methods, including zero. **Türkçe:** Abstract class oluşturan ve extend eden kod yazabilin. Java'da class ve method `abstract` olabilir. Abstract class instantiate edilemez; abstract class reference'ı yalnızca concrete subclass instance'ını gösterebilir. Abstract class sınıf dahil istediği sayıda abstract/non-abstract method içerebilir.

Source page 0329

English: Abstract methods follow all the method override rules and may be defined only within abstract classes. The first concrete subclass of an abstract class must implement all the inherited methods. Abstract classes and methods may not be marked as final. **Türkçe:** Abstract method'lar bütün overriding kurallarına uyar ve yalnızca abstract class içinde tanımlanabilir. Abstract class'ın ilk concrete subclass'ı inherited bütün abstract method'ları implement etmelidir. Abstract class/method `final` olamaz.

English: Create immutable objects. An immutable object is one that cannot be modified after it is declared. An immutable class is commonly implemented with a private constructor, no setter methods, and no ability to modify mutable objects contained within the class. **Türkçe:** Immutable object oluşturun. Immutable object declaration/creation sonrasında değiştirilemez. Yaygın immutable class tasarımında constructor'lar `private` olabilir, setter bulunmaz ve class içindeki mutable object'ler dışarıdan değiştirilemez.

Source page 0330

Review Questions

Türkçe başlık: Review Soruları

English: The answers to the chapter review questions can be found in the Appendix. **Türkçe:** Chapter review sorularının cevaplarını Appendix'te bulabilirsiniz.

English: 1. Which code can be inserted to have the code print 2? **Türkçe:** 1. Kodun 2 yazdırması için hangi kod eklenebilir?

```
public class BirdSeed {
    private int numberBags;
    boolean call;
    public BirdSeed() {
        // LINE 1
        call = false;
        // LINE 2
    }
    public BirdSeed(int numberBags) {
        this.numberBags = numberBags;
    }
    public static void main(String[] args) {
        var seed = new BirdSeed();
        System.out.print(seed.numberBags);
    } }
```

English: A. Replace line 1 with `BirdSeed(2);`. **Türkçe:** A. Line 1'i `BirdSeed(2);` ile değiştirin.

English: B. Replace line 2 with `BirdSeed(2);`. **Türkçe:** B. Line 2'yi `BirdSeed(2);` ile değiştirin.

English: C. Replace line 1 with `new BirdSeed(2);`. **Türkçe:** C. Line 1'i `new BirdSeed(2);` ile değiştirin.

English: D. Replace line 2 with `new BirdSeed(2);`. **Türkçe:** D. Line 2'yi `new BirdSeed(2);` ile değiştirin.

English: E. Replace line 1 with `this(2);`. **Türkçe:** E. Line 1'i `this(2);` ile değiştirin.

English: F. Replace line 2 with `this(2);`. **Türkçe:** F. Line 2'yi `this(2);` ile değiştirin.

English: G. The code prints 2 without any changes. **Türkçe:** G. Kod hiçbir değişiklik yapılmadan 2 yazdırır.

English: 2. Which modifier pairs can be used together in a method declaration? (Choose all that apply.) **Türkçe:** 2. Bir method declaration'ında hangi modifier çiftleri birlikte kullanılabilir? (Uygun olanların tümünü seçin.)

English: A. static and final **Türkçe:** A. static and final

English: B. private and static **Türkçe:** B. private and static

English: C. static and abstract **Türkçe:** C. static and abstract

English: D. private and abstract **Türkçe:** D. private and abstract

English: E. abstract and final **Türkçe:** E. abstract and final

English: F. private and final **Türkçe:** F. private and final

Source page 0331

English: 3. Which of the following statements about methods are true? (Choose all that apply.) **Türkçe:** 3. Method'larla ilgili aşağıdaki ifadelerden hangileri doğrudur? (Uygun olanların tümünü seçin.)

English: A. Overloaded methods must have the same signature. **Türkçe:** A. Overloaded method'lar aynı signature'a sahip olmalıdır.

English: B. Overridden methods must have the same signature. **Türkçe:** B. Overridden method'lar aynı signature'a sahip olmalıdır.

English: C. Hidden methods must have the same signature. **Türkçe:** C. Hidden method'lar aynı signature'a sahip olmalıdır.

English: D. Overloaded methods must have the same return type. **Türkçe:** D. Overloaded method'lar aynı return type'a sahip olmalıdır.

English: E. Overridden methods must have the same return type. **Türkçe:** E. Overridden method'lar aynı return type'a sahip olmalıdır.

English: F. Hidden methods must have the same return type. **Türkçe:** F. Hidden method'lar aynı return type'a sahip olmalıdır.

English: 4. What is the output of the following program? **Türkçe:** 4. Aşağıdaki programın çıktısı nedir?

```
1: class Mammal {  
2: private void sneeze() {}  
3: public Mammal(int age) {  
4: System.out.print("Mammal");  
5: } }
```

```
6: public class Platypus extends Mammal {  
7: int sneeze() { return 1; }  
8: public Platypus() {  
9: System.out.print("Platypus");  
10: }  
11: public static void main(String[] args) {  
12: new Mammal(5);  
13: } }
```

English: A. Platypus **Türkçe:** A. Platypus

English: B. Mammal **Türkçe:** B. Mammal

English: C. PlatypusMammal **Türkçe:** C. PlatypusMammal

English: D. MammalPlatypus **Türkçe:** D. MammalPlatypus

English: E. The code will compile if line 7 is changed. **Türkçe:** E. Line 7 değiştirilirse kod derlenir.

English: F. The code will compile if line 9 is changed. **Türkçe:** F. Line 9 değiştirilirse kod derlenir.

English: 5. Which of the following complete the constructor so that this code prints out 50? (Choose all that apply.) **Türkçe:** 5. Aşağıdakilerden hangileri constructor'ı tamamlayarak kodun 50 yazdırmasını sağlar? (Uygun olanların tümünü seçin.)

```
class Speedster {  
int numSpots;  
}  
public class Cheetah extends Speedster {  
int numSpots;  
public Cheetah(int numSpots) {  
// INSERT CODE HERE  
}
```

Source page 0332

```
public static void main(String[] args) {  
    Speedster s = new Cheetah(50);  
    System.out.print(s.numSpots);  
}  
}
```

English: A. numSpots = numSpots; **Türkçe:** A. numSpots = numSpots;

English: B. numSpots = this.numSpots; **Türkçe:** B. numSpots = this.numSpots;

English: C. this.numSpots = numSpots; **Türkçe:** C. this.numSpots = numSpots;

English: D. numSpots = super.numSpots; **Türkçe:** D. numSpots = super.numSpots;

English: E. super.numSpots = numSpots; **Türkçe:** E. super.numSpots = numSpots;

English: F. The code does not compile regardless of the code inserted into the constructor. **Türkçe:** F. Constructor'a hangi kod eklenirse eklensin kod derlenmez.

English: G. None of the above **Türkçe:** G. Yukarıdakilerin hiçbiri

English: 6. Which of the following declare immutable classes? (Choose all that apply.) **Türkçe:** 6. Aşağıdakilerden hangileri immutable class bildirir? (Uygun olanların tümünü seçin.)

```
public final class Moose {  
    private final int antlers;  
}  
public class Caribou {  
    private int antlers = 10;  
}  
public class Reindeer {  
    private final int antlers = 5;  
}  
public final class Elk {}  
public final class Deer {  
    private final Object o = new Object();  
}
```

English: A. Moose **Türkçe:** A. Moose

English: B. Caribou **Türkçe:** B. Caribou

English: C. Reindeer **Türkçe:** C. Reindeer

English: D. Elk **Türkçe:** D. Elk

English: E. Deer **Türkçe:** E. Deer

English: F. None of the above **Türkçe:** F. Yukarıdakilerin hiçbiri

Source page 0333

English: 7. What is the output of the following code? **Türkçe:** 7. Aşağıdaki kodun çıktısı nedir?

```
1: class Arthropod {
2:     protected void printName(long input) {
3:         System.out.print("Arthropod");
4:     }
5:     void printName(int input) {
6:         System.out.print("Spooky");
7:     } }

8: public class Spider extends Arthropod {
9:     protected void printName(int input) {
10:         System.out.print("Spider");
11:     }
12:     public static void main(String[] args) {
13:         Arthropod a = new Spider();
14:         a.printName((short)4);
15:         a.printName(4);
16:         a.printName(5L);
17:     } }
```

English: A. SpiderSpiderArthropod **Türkçe:** A. SpiderSpiderArthropod

English: B. SpiderSpiderSpider **Türkçe:** B. SpiderSpiderSpider

English: C. SpiderSpookyArthropod **Türkçe:** C. SpiderSpookyArthropod

English: D. SpookySpiderArthropod **Türkçe:** D. SpookySpiderArthropod

English: E. The code will not compile because of line 5. **Türkçe:** E. Kod line 5 nedeniyle derlenmez.

English: F. The code will not compile because of line 9. **Türkçe:** F. Kod line 9 nedeniyle derlenmez.

English: G. None of the above **Türkçe:** G. Yukarıdakilerin hiçbiri

English: 8. What is the result of the following code? **Türkçe:** 8. Aşağıdaki kodun sonucu nedir?

```
1: abstract class Bird {
2: private final void fly() { System.out.println("Bird"); }
3: protected Bird() { System.out.print("Wow- "); }
4: }
5: public class Pelican extends Bird {
6: public Pelican() { System.out.print("Oh- "); }
7: protected void fly() { System.out.println("Pelican"); }
8: public static void main(String[] args) {
9: var chirp = new Pelican();
10: chirp.fly();
11: } }
```

Source page 0334

English: A. Oh- Bird **Türkçe:** A. Oh- Bird

English: B. Oh- Pelican **Türkçe:** B. Oh- Pelican

English: C. Wow- Oh- Bird **Türkçe:** C. Wow- Oh- Bird

English: D. Wow- Oh- Pelican **Türkçe:** D. Wow- Oh- Pelican

English: E. The code contains a compilation error. **Türkçe:** E. Kod bir compilation error içerir.

English: F. None of the above **Türkçe:** F. Yukarıdakilerin hiçbiri

English: 9. Which of the following statements about overridden methods are true? (Choose all that apply.)

Türkçe: 9. Overridden method'larla ilgili aşağıdaki ifadelerden hangileri doğrudur? (Uygun olanların tümünü seçin.)

English: A. An overridden method must contain method parameters that are the same or covariant with the method parameters in the inherited method. **Türkçe:** A. Overridden method, inherited method'daki parameter'larla aynı veya covariant method parameter'ları içermelidir.

English: B. An overridden method may declare a new exception, provided it is not checked. **Türkçe:** B. Overridden method, checked exception olmaması koşuluyla yeni bir exception bildirebilir.

English: C. An overridden method must be more accessible than the method in the parent class. **Türkçe:** C. Overridden method, parent class'taki method'dan daha accessible olmalıdır.

English: D. An overridden method may declare a broader checked exception than the method in the parent class. **Türkçe:** D. Overridden method, parent class'taki method'dan daha broad bir checked exception bildirebilir.

English: E. If an inherited method returns void, then the overridden version of the method must return void. **Türkçe:** E. Inherited method `void` döndürüyorsa method'un overridden sürümü de `void` döndürmelidir.

English: F. None of the above **Türkçe:** F. Yukarıdakilerin hiçbiri

English: 10. Which of the following pairs, when inserted into the blanks, allow the code to compile? (Choose all that apply.) **Türkçe:** 10. Aşağıdaki çiftlerden hangileri boşluklara eklendiğinde kodun derlenmesini sağlar? (Uygun olanların tümünü seçin.)

```
1: public class Howler {
2:     public Howler(long shadow) {
3:     ;
4: }
5: private Howler(int moon) {
6:     super();
7: }
8: }
9: class Wolf extends Howler {
10:     protected Wolf(String stars) {
11:         super(2L);
12:     }
13:     public Wolf() {
14:     ;
15: }
16: }
```

English: A. `this(3)` at line 3, `this("")` at line 14 **Türkçe:** A. Line 3'te `this(3)`, line 14'te `this("")`

English: B. `this()` at line 3, `super(1)` at line 14 **Türkçe:** B. Line 3'te `this()`, line 14'te `super(1)`

English: C. `this((short)1)` at line 3, `this(null)` at line 14 **Türkçe:** C. Line 3'te `this((short)1)`, line 14'te `this(null)`

English: D. `super()` at line 3, `super()` at line 14 **Türkçe:** D. Line 3'te `super()`, line 14'te `super()`

Source page 0335

English: E. `this(2L)` at line 3, `super((short)2)` at line 14 **Türkçe:** E. Line 3'te `this(2L)`, line 14'te `super((short)2)`

English: F. `this(5)` at line 3, `super(null)` at line 14 **Türkçe:** F. Line 3'te `this(5)`, line 14'te `super(null)`

English: G. Remove lines 3 and 14. **Türkçe:** G. Line 3 ve 14'ü kaldırın.

English: 11. What is the result of the following? **Türkçe:** 11. Aşağıdakilerin sonucu nedir?

```
1: public class PolarBear {
2:     StringBuilder value = new StringBuilder("t");
3:     { value.append("a"); }
4:     { value.append("c"); }
5:     private PolarBear() {
6:         value.append("b");
7:     }
8:     public PolarBear(String s) {
9:         this();
10:        value.append(s);
11:    }
12:    public PolarBear(CharSequence p) {
13:        value.append(p);
14:    }
15:    public static void main(String[] args) {
16:        Object bear = new PolarBear();
17:        bear = new PolarBear("f");
18:        System.out.println(((PolarBear)bear).value);
19:    } }
```

English: A. tacb **Türkçe:** A. tacb

English: B. tacf **Türkçe:** B. tacf

English: C. tacbf **Türkçe:** C. tacbf

English: D. tcafb **Türkçe:** D. tcafb

English: E. taftacb **Türkçe:** E. taftacb

English: F. The code does not compile. **Türkçe:** F. Kod derlenmez.

English: G. An exception is thrown. **Türkçe:** G. Bir exception fırlatılır.

English: 12. How many lines of the following program contain a compilation error? **Türkçe:** 12. Aşağıdaki programın kaç satırında compilation error vardır?

```
1: public class Rodent {
2:     public Rodent(Integer x) {}
3:     protected static Integer chew() throws Exception {
4:         System.out.println("Rodent is chewing");
5:         return 1;
6:     }
7: }
```

Source page 0336

```
8: class Beaver extends Rodent {  
9: public Number chew() throws RuntimeException {  
10: System.out.println("Beaver is chewing on wood");  
11: return 2;  
12: } }
```

English: A. None **Türkçe:** A. Hiçbiri

English: B. 1 **Türkçe:** B. 1

English: C. 2 **Türkçe:** C. 2

English: D. 3 **Türkçe:** D. 3

English: E. 4 **Türkçe:** E. 4

English: F. 5 **Türkçe:** F. 5

English: 13. Which of these classes compile and will include a default constructor created by the compiler? (Choose all that apply.) **Türkçe:** 13. Bu class'lardan hangileri derlenir ve compiler tarafından oluşturulan default constructor'ı içerir? (Uygun olanların tümünü seçin.)

English: A. **Türkçe:** A.

```
public class Bird {}
```

English: B. **Türkçe:** B.

```
public class Bird {  
public bird() {}  
}
```

English: C. **Türkçe:** C.

```
public class Bird {  
public bird(String name) {}  
}
```

English: D. **Türkçe:** D.

```
public class Bird {  
    public Bird() {}  
}
```

English: E. **Türkçe:** E.

```
public class Bird {  
    Bird(String name) {}  
}
```

English: F. **Türkçe:** F.

```
public class Bird {  
    private Bird(int age) {}  
}
```

Source page 0337

English: G. **Türkçe:** G.

```
public class Bird {  
    public Bird bird() { return null; }  
}
```

English: 14. Which of the following statements about inheritance are correct? (Choose all that apply.)

Türkçe: 14. Inheritance ile ilgili aşağıdaki ifadelerden hangileri doğrudur? (Uygun olanların tümünü seçin.)

English: A. A class can directly extend any number of classes. **Türkçe:** A. Bir class doğrudan istediği sayıda class'ı extend edebilir.

English: B. A class can implement any number of interfaces. **Türkçe:** B. Bir class istediği sayıda interface'i implement edebilir.

English: C. All variables inherit java.lang.Object. **Türkçe:** C. Bütün variable'lar `java.lang.Object`'ten inherit eder.

English: D. If class A is extended by B, then B is a superclass of A. **Türkçe:** D. Class A, B tarafından extend edilirse B, A'nın superclass'ıdır.

English: E. If class C implements interface D, then C is a subtype of D. **Türkçe:** E. Class C, D interface'ini implement ederse C, D'nin subtype'ıdır.

English: F. Multiple inheritance is the property of a class to have multiple direct superclasses. **Türkçe:** F. Multiple inheritance, bir class'ın birden fazla doğrudan superclass'a sahip olması özelliğidir.

English: 15. Which statements about the following program are correct? (Choose all that apply.) **Türkçe:** 15. Aşağıdaki programla ilgili hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

```
1: abstract class Nocturnal {
2:     boolean isBlind();
3: }
4: public class Owl extends Nocturnal {
5:     public boolean isBlind() { return false; }
6:     public static void main(String[] args) {
7:         var nocturnal = (Nocturnal)new Owl();
8:         System.out.println(nocturnal.isBlind());
9:     } }
```

English: A. It compiles and prints true. **Türkçe:** A. Derlenir ve `true` yazdırır.

English: B. It compiles and prints false. **Türkçe:** B. Derlenir ve `false` yazdırır.

English: C. The code will not compile because of line 2. **Türkçe:** C. Kod line 2 nedeniyle derlenmez.

English: D. The code will not compile because of line 5. **Türkçe:** D. Kod line 5 nedeniyle derlenmez.

English: E. The code will not compile because of line 7. **Türkçe:** E. Kod line 7 nedeniyle derlenmez.

English: F. The code will not compile because of line 8. **Türkçe:** F. Kod line 8 nedeniyle derlenmez.

English: G. None of the above **Türkçe:** G. Yukarıdakilerin hiçbiri

English: 16. What is the result of the following? **Türkçe:** 16. Aşağıdakilerin sonucu nedir?

```
1: class Arachnid {
2:     static StringBuilder sb = new StringBuilder();
3:     { sb.append("c"); }
4:     static
5:     { sb.append("u"); }
6:     { sb.append("r"); }
7: }
```

Source page 0338

```
8: public class Scorpion extends Arachnid {
9: static
10: { sb.append("q"); }
11: { sb.append("m"); }
12: public static void main(String[] args) {
13: System.out.print(Scorpion.sb + " ");
14: System.out.print(Scorpion.sb + " ");
15: new Arachnid();
16: new Scorpion();
17: System.out.print(Scorpion.sb);
18: } }
```

English: A. qu qu qumrcrc **Türkçe:** A. qu qu qumrcrc

English: B. u u ucrcrm **Türkçe:** B. u u ucrcrm

English: C. uq uq uqmcrcr **Türkçe:** C. uq uq uqmcrcr

English: D. uq uq uqrcrm **Türkçe:** D. uq uq uqrcrm

English: E. qu qu qumrcrc **Türkçe:** E. qu qu qumrcrc

English: F. qu qu qucrmr **Türkçe:** F. qu qu qucrmr

English: G. The code does not compile. **Türkçe:** G. Kod derlenmez.

English: 17. Which of the following are true? (Choose all that apply.) **Türkçe:** 17. Aşağıdakilerden hangileri doğrudur? (Uygun olanların tümünü seçin.)

English: A. this() can be called from anywhere in a constructor. **Türkçe:** A. `this()` bir constructor'ın herhangi bir yerinden çağrılabilir.

English: B. this() can be called from anywhere in an instance method. **Türkçe:** B. `this()` bir instance method'un herhangi bir yerinden çağrılabilir.

English: C. this.variableName can be called from any instance method in the class. **Türkçe:** C. `this.variableName`, class'taki herhangi bir instance method'dan kullanılabilir.

English: D. this.variableName can be called from any static method in the class. **Türkçe:** D. `this.variableName`, class'taki herhangi bir static method'dan kullanılabilir.

English: E. You can call the default constructor written by the compiler using this(). **Türkçe:** E. Compiler'ın eklediği default constructor `this()` kullanılarak çağrılabilir.

English: F. You can access a private constructor with the main() method in the same class. **Türkçe:** F. Aynı class içindeki `main()` method'u ile `private` constructor'a erişilebilir.

English: 18. Which statements about the following classes are correct? (Choose all that apply.) **Türkçe:** 18. Aşağıdaki class'larla ilgili hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

```
1: public class Mammal {
2:     private void eat() {}
3:     protected static void drink() {}
4:     public Integer dance(String p) { return null; }
5: }
6: class Primate extends Mammal {
7:     public void eat(String p) {}
8: }
9: class Monkey extends Primate {
```

Source page 0339

```
10: public static void drink() throws RuntimeException {}
11: public Number dance(CharSequence p) { return null; }
12: public int eat(String p) {}
13: }
```

English: A. The eat() method in Mammal is correctly overridden on line 7. **Türkçe:** A. `Mammal` içindeki `eat()` method'u line 7'de doğru biçimde override edilmiştir.

English: B. The eat() method in Mammal is correctly overloaded on line 7. **Türkçe:** B. `Mammal` içindeki `eat()` method'u line 7'de doğru biçimde overload edilmiştir.

English: C. The drink() method in Mammal is correctly overridden on line 10. **Türkçe:** C. `Mammal` içindeki `drink()` method'u line 10'da doğru biçimde override edilmiştir.

English: D. The drink() method in Mammal is correctly hidden on line 10. **Türkçe:** D. `Mammal` içindeki `drink()` method'u line 10'da doğru biçimde hide edilmiştir.

English: E. The dance() method in Mammal is correctly overridden on line 11. **Türkçe:** E. `Mammal` içindeki `dance()` method'u line 11'de doğru biçimde override edilmiştir.

English: F. The dance() method in Mammal is correctly overloaded on line 11. **Türkçe:** F. `Mammal` içindeki `dance()` method'u line 11'de doğru biçimde overload edilmiştir.

English: G. The eat() method in Primate is correctly hidden on line 12. **Türkçe:** G. `Primate` içindeki `eat()` method'u line 12'de doğru biçimde hide edilmiştir.

English: H. The eat() method in Primate is correctly overloaded on line 12. **Türkçe:** H. `Primate` içindeki `eat()` method'u line 12'de doğru biçimde overload edilmiştir.

English: 19. What is the output of the following code? **Türkçe:** 19. Aşağıdaki kodun çıktısı nedir?

```
1: class Reptile {
2: {System.out.print("A");}
3: public Reptile(int hatch) {}
4: void layEggs() {
5: System.out.print("Reptile");
6: } }
7: public class Lizard extends Reptile {
8: static {System.out.print("B");}
9: public Lizard(int hatch) {}
10: public final void layEggs() {
11: System.out.print("Lizard");
12: }
13: public static void main(String[] args) {
14: var reptile = new Lizard(1);
15: reptile.layEggs();
16: } }
```

English: A. AALizard **Türkçe:** A. AALizard

English: B. BALizard **Türkçe:** B. BALizard

English: C. BLizardA **Türkçe:** C. BLizardA

English: D. ALizard **Türkçe:** D. ALizard

English: E. The code will not compile because of line 3. **Türkçe:** E. Kod line 3 nedeniyle derlenmez.

English: F. None of the above **Türkçe:** F. Yukarıdakilerin hiçbiri

Source page 0340

English: 20. Which statement about the following program is correct? **Türkçe:** 20. Aşağıdaki programla ilgili hangi ifade doğrudur?

```
1: class Bird {
2:   int feathers = 0;
3:   Bird(int x) { this.feathers = x; }
4:   Bird fly() {
5:     return new Bird(1);
6:   } }
7:   class Parrot extends Bird {
8:     protected Parrot(int y) { super(y); }
9:     protected Parrot fly() {
10:      return new Parrot(2);
11:    } }
12:   public class Macaw extends Parrot {
13:     public Macaw(int z) { super(z); }
14:     public Macaw fly() {
15:      return new Macaw(3);
16:    }
17:     public static void main(String... sing) {
18:       Bird p = new Macaw(4);
19:       System.out.print(((Parrot)p.fly()).feathers);
20:     } }
```

English: A. One line contains a compiler error. **Türkçe:** A. Bir satır compiler error içerir.

English: B. Two lines contain compiler errors. **Türkçe:** B. İki satır compiler error içerir.

English: C. Three lines contain compiler errors. **Türkçe:** C. Üç satır compiler error içerir.

English: D. The code compiles but throws a `ClassCastException` at runtime. **Türkçe:** D. Kod derlenir ancak runtime'da `ClassCastException` fırlatır.

English: E. The program compiles and prints 3. **Türkçe:** E. Program derlenir ve 3 yazdırır.

English: F. The program compiles and prints 0. **Türkçe:** F. Program derlenir ve 0 yazdırır.

English: 21. Which of the following are properties of immutable classes? (Choose all that apply.) **Türkçe:** 21. Aşağıdakilerden hangileri immutable class özellikleridir? (Uygun olanların tümünü seçin.)

English: A. The class can contain setter methods, provided they are marked final. **Türkçe:** A. `final` olarak işaretlenmeleri koşuluyla class setter method'lar içerebilir.

English: B. The class must not be able to be extended outside the class declaration. **Türkçe:** B. Class declaration'ı dışında bu class'ı extend etmek mümkün olmamalıdır.

English: C. The class may not contain any instance variables. **Türkçe:** C. Class hiçbir instance variable içeremez.

English: D. The class must be marked static. **Türkçe:** D. Class `static` olarak işaretlenmelidir.

English: E. The class may not contain any static variables. **Türkçe:** E. Class hiçbir static variable içeremez.

English: F. The class may only contain private constructors. **Türkçe:** F. Class yalnızca `private` constructor'lar içerebilir.

English: G. The data for mutable instance variables may be read, provided they cannot be modified by the caller. **Türkçe:** G. Çağırın kod tarafından değiştirilememesi koşuluyla mutable instance variable'ların verileri okunabilir.

Source page 0341

English: 22. What does the following program print? **Türkçe:** 22. Aşağıdaki program ne yazdırır?

```
1: class Person {
2: static String name;
3: void setName(String q) { name = q; } }
4: public class Child extends Person {
5: static String name;
6: void setName(String w) { name = w; }
7: public static void main(String[] p) {
8: final Child m = new Child();
9: final Person t = m;
10: m.name = "Elysia";
11: t.name = "Sophia";
12: m.setName("Webby");
13: t.setName("Olivia");
14: System.out.println(m.name + " " + t.name);
15: } }
```

English: A. Elysia Sophia **Türkçe:** A. Elysia Sophia

English: B. Webby Olivia **Türkçe:** B. Webby Olivia

English: C. Olivia Olivia **Türkçe:** C. Olivia Olivia

English: D. Olivia Sophia **Türkçe:** D. Olivia Sophia

English: E. The code does not compile. **Türkçe:** E. Kod derlenmez.

English: F. None of the above **Türkçe:** F. Yukarıdakilerin hiçbiri

English: 23. What is the output of the following program? **Türkçe:** 23. Aşağıdaki programın çıktısı nedir?

```
1: class Canine {
2: public Canine(boolean t) { logger.append("a"); }
3: public Canine() { logger.append("q"); }
4:
5: private StringBuilder logger = new StringBuilder();
6: protected void print(String v) { logger.append(v); }
7: protected String view() { return logger.toString(); }
8: }
9:
10: class Fox extends Canine {
11: public Fox(long x) { print("p"); }
12: public Fox(String name) {
13: this(2);
14: print("z");
15: }
```

Source page 0342

```
16: }
17:
18: public class Fennec extends Fox {
19: public Fennec(int e) {
20: super("tails");
21: print("j");
22: }
23: public Fennec(short f) {
24: super("eevee");
25: print("m");
26: }
27:
28: public static void main(String... unused) {
29: System.out.println(new Fennec(1).view());
30: } }
```

English: A. qpz **Türkçe:** A. qpz

English: B. qpzj **Türkçe:** B. qpzj

English: C. jzpa **Türkçe:** C. jzpa

English: D. apj **Türkçe:** D. apj

English: E. apjm **Türkçe:** E. apjm

English: F. The code does not compile. **Türkçe:** F. Kod derlenmez.

English: G. None of the above **Türkçe:** G. Yukarıdakilerin hiçbiri

English: 24. What is printed by the following program? **Türkçe:** 24. Aşağıdaki program ne yazdırır?

```
1: class Antelope {
2: public Antelope(int p) {
3: System.out.print("4");
4: }
5: { System.out.print("2"); }
6: static { System.out.print("1"); }
7: }
8: public class Gazelle extends Antelope {
9: public Gazelle(int p) {
10: super(6);
11: System.out.print("3");
12: }
```

Source page 0343

```
13: public static void main(String hopping[]) {
14: new Gazelle(0);
15: }
16: static { System.out.print("8"); }
17: { System.out.print("9"); }
18: }
```

English: A. 182640 **Türkçe:** A. 182640

English: B. 182943 **Türkçe:** B. 182943

English: C. 182493 **Türkçe:** C. 182493

English: D. 421389 **Türkçe:** D. 421389

English: E. The code does not compile. **Türkçe:** E. Kod derlenmez.

English: F. The output cannot be determined until runtime. **Türkçe:** F. Çıktı runtime'a kadar belirlenemez.

English: 25. Which of the following are true about a concrete class? (Choose all that apply.) **Türkçe:** 25. Concrete class ile ilgili aşağıdakilerden hangileri doğrudur? (Uygun olanların tümünü seçin.)

English: A. A concrete class can be declared as abstract. **Türkçe:** A. Concrete class **abstract** olarak bildirilebilir.

English: B. A concrete class must implement all inherited abstract methods. **Türkçe:** B. Concrete class inherited bütün abstract method'ları implement etmelidir.

English: C. A concrete class can be marked as final. **Türkçe:** C. Concrete class **final** olarak işaretlenebilir.

English: D. A concrete class must be immutable. **Türkçe:** D. Concrete class immutable olmalıdır.

English: E. A concrete method that implements an abstract method must match the method declaration of the abstract method exactly. **Türkçe:** E. Abstract method'u implement eden concrete method, abstract method'un declaration'ıyla tamamen eşleşmelidir.

English: 26. What is the output of the following code? **Türkçe:** 26. Aşağıdaki kodun çıktısı nedir?

```
4: public abstract class Whale {  
5: public abstract void dive();  
6: public static void main(String[] args) {  
7: Whale whale = new Orca();  
8: whale.dive(3);  
9: }  
10: }  
11: class Orca extends Whale {  
12: static public int MAX = 3;  
13: public void dive() {  
14: System.out.println("Orca diving");  
15: }  
16: public void dive(int... depth) {  
17: System.out.println("Orca diving deeper "+MAX);  
18: } }
```

Source page 0344

English: A. Orca diving **Türkçe:** A. Orca diving

English: B. Orca diving deeper 3 **Türkçe:** B. Orca diving deeper 3

English: C. The code will not compile because of line 4. **Türkçe:** C. Kod line 4 nedeniyle derlenmez.

English: D. The code will not compile because of line 8. **Türkçe:** D. Kod line 8 nedeniyle derlenmez.

English: E. The code will not compile because of line 11. **Türkçe:** E. Kod line 11 nedeniyle derlenmez.

English: F. The code will not compile because of line 12. **Türkçe:** F. Kod line 12 nedeniyle derlenmez.

English: G. The code will not compile because of line 17. **Türkçe:** G. Kod line 17 nedeniyle derlenmez.

English: H. None of the above **Türkçe:** H. Yukarıdakilerin hiçbiri

Page coverage ledger

Source page	Marker
0275	<!-- source-page: 0275 -->
0276	<!-- source-page: 0276 -->
0277	<!-- source-page: 0277 -->
0278	<!-- source-page: 0278 -->
0279	<!-- source-page: 0279 -->
0280	<!-- source-page: 0280 -->
0281	<!-- source-page: 0281 -->
0282	<!-- source-page: 0282 -->
0283	<!-- source-page: 0283 -->
0284	<!-- source-page: 0284 -->
0285	<!-- source-page: 0285 -->
0286	<!-- source-page: 0286 -->
0287	<!-- source-page: 0287 -->
0288	<!-- source-page: 0288 -->
0289	<!-- source-page: 0289 -->
0290	<!-- source-page: 0290 -->
0291	<!-- source-page: 0291 -->
0292	<!-- source-page: 0292 -->
0293	<!-- source-page: 0293 -->
0294	<!-- source-page: 0294 -->
0295	<!-- source-page: 0295 -->
0296	<!-- source-page: 0296 -->
0297	<!-- source-page: 0297 -->
0298	<!-- source-page: 0298 -->
0299	<!-- source-page: 0299 -->
0300	<!-- source-page: 0300 -->
0301	<!-- source-page: 0301 -->
0302	<!-- source-page: 0302 -->
0303	<!-- source-page: 0303 -->
0304	<!-- source-page: 0304 -->
0305	<!-- source-page: 0305 -->

Source page	Marker
0306	<!-- source-page: 0306 -->
0307	<!-- source-page: 0307 -->
0308	<!-- source-page: 0308 -->
0309	<!-- source-page: 0309 -->
0310	<!-- source-page: 0310 -->
0311	<!-- source-page: 0311 -->
0312	<!-- source-page: 0312 -->
0313	<!-- source-page: 0313 -->
0314	<!-- source-page: 0314 -->
0315	<!-- source-page: 0315 -->
0316	<!-- source-page: 0316 -->
0317	<!-- source-page: 0317 -->
0318	<!-- source-page: 0318 -->
0319	<!-- source-page: 0319 -->
0320	<!-- source-page: 0320 -->
0321	<!-- source-page: 0321 -->
0322	<!-- source-page: 0322 -->
0323	<!-- source-page: 0323 -->
0324	<!-- source-page: 0324 -->
0325	<!-- source-page: 0325 -->
0326	<!-- source-page: 0326 -->
0327	<!-- source-page: 0327 -->
0328	<!-- source-page: 0328 -->
0329	<!-- source-page: 0329 -->
0330	<!-- source-page: 0330 -->
0331	<!-- source-page: 0331 -->
0332	<!-- source-page: 0332 -->
0333	<!-- source-page: 0333 -->
0334	<!-- source-page: 0334 -->
0335	<!-- source-page: 0335 -->
0336	<!-- source-page: 0336 -->
0337	<!-- source-page: 0337 -->
0338	<!-- source-page: 0338 -->

Source page	Marker
0339	<!-- source-page: 0339 -->
0340	<!-- source-page: 0340 -->
0341	<!-- source-page: 0341 -->
0342	<!-- source-page: 0342 -->
0343	<!-- source-page: 0343 -->
0344	<!-- source-page: 0344 -->

Appendix · Kaynak dışı teknik pekiştirme

Kaynak bölümün çevirisine ait olmayan önceki OCP pekiştirme içeriği ayrı ana kaynakta korunmuştur: [Unit 06 technical memory notes](#).